

Geodesic mesh and continuum variable solver in SPECTRUM

Vladimir Florinski

25 January, 2025

1 Introduction

SPECTRUM stands for Space Plasma and Energetic Charged particle TRansport on Unstructured Meshes. This document is mainly concerned with the “space plasma” and “unstructured mesh” elements. SPECTRUM is based on a geodesic mesh, a construct widely used in geosciences because it is adapted to the surface of the sphere (Earth in that case) and provides a nearly homogeneous coverage of the sphere, in contrast to polar (r, θ, φ) meshes that have geometry singularities at the poles ($\theta = 0, \pi$). Geodesic meshes offer important advantages, such as uniform coverage and less severe time step constraints at the expense of increased code complexity. A geodesic mesh shares properties with both structured and unstructured meshes, but is generally classified as unstructured. The manual goes into significant depth explaining how the mesh is generated and how the variables defined on this mesh are processed. Because of SPECTRUM’s highly modular architecture, each functional unit is represented by a C++ class. Different sections of the document deal with different types of classes, such as those operating on the entire sphere, or on its sub-domain called a block. The information presented here will be of help to developers who wish to improve some aspects of the continuum model, such as adding higher order reconstruction or implementing different conservation laws.

2 Partitioning a sphere into tiles

A spherical *tesselation* is a partition of a surface of a sphere (usually assumed to have a unit radius) into spherical polygons. It is an example of an unstructured mesh, where each polygon must be addressed individually rather than through a pair of indices. A tree numbering scheme also exists, but it is typically too cumbersome to apply for frequent operations and is only used in the mesh refinement context. A tessellation is obtained by choosing an initial polyhedron, projecting its edges to a sphere and recursively dividing each edge in half to form smaller and smaller polygons. The SPECTRUM software offers a choice of 5 regular polyhedra (Platonic solids) defined in the header file `polyhedron.hh`: tetrahedron, hexahedron (cube), octahedron, dodecahedron, and icosahedron. The tetrahedron and the octahedron produce meshes with a high degree of non-uniformity, and these shapes are not recommended for production use. All files discussed in this section reside in the `geodesic` sub-directory of the SPECTRUM source tree.

A tessellation object can be used in two ways. First, it can represent the entire computational mesh on the surface of a sphere. This is acceptable if domain decomposition is performed only in the radial direction by sub-dividing the computational domain into *slabs* (annular regions consisting of one or more spherical *shells* or zone layers). In this case a tessellation provides the compute portion of the code with complete information about the mesh, including the coordinates of the vertices and/or face centers and the *connectivity tables* describing the relationship between neighbor elements. However, this type of parallelization is limited to typically ~200–300 cores (corresponding to two shells per core). To improve parallelization properties a decomposition must be also performed on the sphere. In this case a computational unit, known as a *grid block*, is a tensor product of a spherical *sector* (large polygon sub-divided into individual faces) and a slab. In that configuration, which is adopted in SPECTRUM, a tessellation object is used in an auxiliary role by providing the geometry and neighbor information to grid blocks, which keep their own, local versions of the coordinate lists and the element connectivity tables. This way a tessellation object is needed only in the initialization stage, and can be unloaded afterwards, unless a location service is required, for example in post-processing.

2.1 Defining the base polyhedron

The base tessellations class is called **Schlafl**i. Its definition and implementation can be found in the header file **polyhedron.hh**. The class defines the Schläfli mathematical symbol $\{p, q\}$, where p is the number of vertices per face, and q is the number of edges that meet at every vertex. The formulas for the total number of vertices (N_v^0), edges (N_e^0), and faces (N_f^0) in a polyhedron are

$$N_v^0 = \frac{4p}{4 - (p-2)(q-2)}, \quad (1)$$

$$N_e^0 = \frac{2pq}{4 - (p-2)(q-2)}, \quad (2)$$

$$N_f^0 = \frac{4q}{4 - (p-2)(q-2)}. \quad (3)$$

The three quantities so defined satisfy the Euler formula,

$$N_v + N_f - N_e = 2. \quad (4)$$

Here and below a superscript is used to indicate the *division* (see below). The **Schlafl**i class is not intended for instantiation by the user.

The derived class template **Polyhedron** takes one template argument of type **int**, which corresponds to the number of faces in the base polyhedron, from **POLY_TETRAHEDRON** (4) to **POLY_ICOSAHEDRON** (20). The class provides coordinates (latitude-longitude) of the vertices of the polyhedron and two neighbor connectivity arrays: vertex to vertex (**vert_vert**) and face to vertex (**face_vert**). Figure 1 shows all five base polyhedra projected to a sphere. The code in **polyhedron.hh** shows the face numbering in the comments. There is typically no need to create an instance of the **Polyhedron** class in the user code.

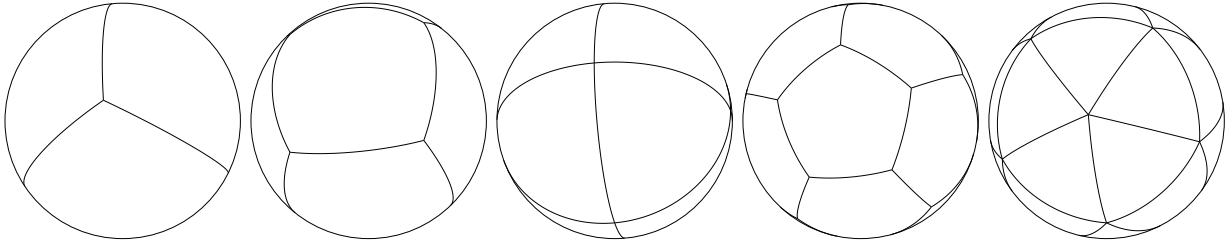


Figure 1: Five regular polyhedra, projected onto a sphere, that can be used as templates for a spherical tessellation, from left to right: tetrahedron, hexahedron, octahedron, dodecahedron, icosahedron.

2.2 Recursive division of the base polyhedron

The class template **SphericalTessellation** is derived from **Polyhedron**; it adds **max_division** as an integer parameter to designate the highest division of the mesh that the class can handle (larger numbers require more memory). Division 0 corresponds to the original polyhedron, while each subsequent division is created by a recursive bisection procedure. This class includes 8 connectivity tables, vertex to vertex (**vv_con**), vertex to edge (**ve_con**), etc., all the way to **ff_con** (there is no edge to edge table.) All neighbor connectivity tables are ordered in the right hand sense, i.e., counter-clockwise (with the axis pointing outward), to simplify mesh traversal. Cartesian coordinates of the vertices (**vert_cart**) are provided. A process is expected to have one and only one instance of the tessellation, so a copy constructor for the class is deleted.

A tessellation consists of either triangular or quadrilateral shapes. Tetrahedral, octahedral, dodecahedral (except division 0), and icosahedral meshes are triangular, while the hexahedral mesh is quadrilateral. The dodecahedral tessellation present an additional complication because the division 0 faces are pentagons while at all other divisions the faces are triangular. For this reason in the first dodecahedral division each parent face is divided into five daughter faces, rather than four (this is controlled by the parameter **children_per_face**).

The following recursive expressions give the number of elements in a division d given their counts in the parent division $d - 1$:

$$N_v^d = \begin{cases} N_v^{d-1} + N_f^{d-1} & (\text{dodecahedral mesh for } d = 1), \\ N_v^{d-1} + N_e^{d-1} & (\text{triangular meshes}), \\ N_v^{d-1} + N_e^{d-1} + N_f^{d-1} & (\text{quad meshes}), \end{cases} \quad (5)$$

$$N_f^d = N_c^{d-1} N_f^{d-1}, \quad (6)$$

where N_c^{d-1} is the number of division d children of a face at division $d - 1$ (4 or 5). The number of edges is computed using the Euler formula (4). The class `SphericalTessellation` is declared in the header file `spherical_tessellation.hh`, and its non-inlined functions are defined in `spherical_tessellation.cc`.

The mesh construction process is performed by the member method `ComputeAll()`. In what follows the first entry in the face to vertex connectivity table for a face is called the *base vertex* of that face. At each division (1,2,3, etc.) 8 functions are called (each with a single argument `div`, which is the division of the table to be generated). The sequence is as follows:

1. New vertices are inserted at edge midpoints of the parent division. For the hexahedral mesh at all divisions and for the dodecahedral mesh at division 1, an additional vertex is also inserted at the center of each parent face; this is specified by the boolean member arrays `newverts_at_face` and `newverts_at_edge`; the values can be different at divisions 0 and higher divisions. The center is obtained as an intersection point of two great circles passing through vertices 0,2 and 1,3. Each new vertex in the mesh has a unique index that is not division-specific because a set of vertices at any division also includes all parent vertex sets. Conversely, the numbering of faces and edges is specific to each division. Thus vertices between 0 and $N_v - 1$ are known to be the vertices of the template polyhedron; these will be later referred to as *singular corners* where *mesh defects* occur. After the new vertices are added, the code calculates their neighbors (VV table) in the mesh. All of this is performed in `RefineVert()`. The function is *not* used during division 0 mesh processing because the base polyhedron VV connectivity is hard coded in the `Polyhedron` class.

2. Edges are created anew for each division based on vertex to vertex connectivity. One edge is inserted for each unique connection in the VV array. Because there are always *two* connections ($v_1 \rightarrow v_2$ and $v_2 \rightarrow v_1$) an edge is inserted only if the corresponding indices satisfy $I_{v1} > I_{v2}$. The edge to vertex array (two entries per edge) is filled here as well. These functions are performed in `AddEdges()`.

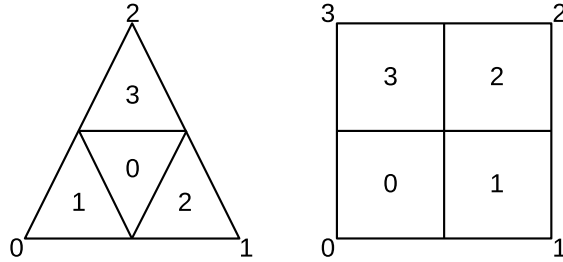


Figure 2: The numbering principle for the daughter faces obtained by a recursive division of triangular (left) and quadrilateral (right) parent faces.

3. The code calculates the new face topology. As a face is divided into 4 (or five) faces, one of the daughter faces (face 0 in Figure 2) inherits the index of the parent, and the rest get sequential numbers that are higher than the number of faces in the parent division. The function `RefineFace()` performs the division and also computes the face to vertex connectivity table based on the FE table of the parent division. The function is *not* used during division 0 mesh processing because the FV table is encoded in `Polyhedron`.

4. Vertex to edge neighbor arrays are computed in `VertEdgeConn()`. This is just an inverse of the EV array generated earlier in Step 2. The file `spherical_tessellation.cc` implements a generic function called `BuildReverse()` that builds a connectivity table from nodes of type 2 to nodes of type 1 that is the inverse of a given connectivity table, from nodes of type 1 to nodes of type 2 (where “nodes” could be vertices, edges, etc.) This is accomplished by traversing the type 1 to type 2 table and for each type 2 node in the

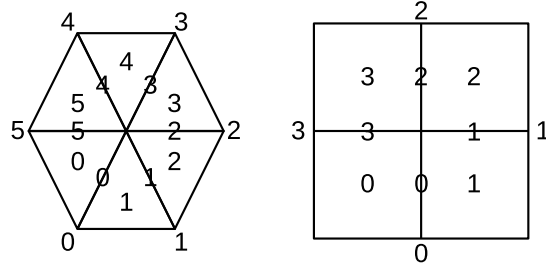


Figure 3: Numbering of neighbor elements of a vertex for triangular (left) and quadrilateral (right) meshes.

table *appending* that type 1 node's index to its list of neighbors until all positions are filled. The code does not use C++ containers (in the hopes of porting to CUDA/HIP which lack that functionality), so the last unfilled position must be stored separately for each node. The resulting table, of course, is not CC-ordered, so `VertEdgeConn()` performs an additional ordering loop with the help of the VV neighbor array computed previously. To find an element in an array of integers the code employs the function `InList()`, which is defined in the header `common/definitions.hh`.

5. Next, the vertex to face connectivity is computed. Because FV is already available, the required table can be obtained as the inverse. Counter-clockwise sorting is again performed, so that by the end of this step VV, VE, and VF connectivity tables are fully synchronized. Figure 3 shows the numbering schemes for vertex, edge, and face neighbors of a vertex. This step is performed by the member function `VertFaceConn()`.

6. The routine `EdgeFaceConn()` calculates the face neighbors for each edge. This is done by finding two faces that share *both* of that edge's vertices (from the EV table). A handy helper function `Match2vtf` is provided specifically to perform this matching.

7. The face to edge connectivity is calculated in `FaceEdgeConn()`. The construction is based on finding an edge connecting each pair of vertices that belong to that face using the available FV table; the matching is done by the helper routine `Match2ve()`.

8. The final step is generating the face to face connection table which is the responsibility of the function `FaceFaceConn()`. The numbering principle is depicted in Figure 4. Everything is ordered relative to the base vertex of the face. Because the FE neighbor table is already available, it is a simple matter to use it to figure out the corresponding face neighbor.

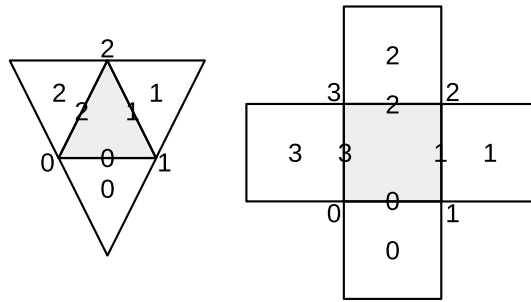


Figure 4: Numbering of neighbor elements of a face for triangular (left) and quadrilateral (right) meshes.

The public interface of `SphericalTessellation` is extremely simple. All of the work is performed in the constructor. Three additional public routines are provided to return the number of elements at a specific division, `NVerts()`, `NEdges()`, and `NFaces()`.

2.3 Visualizing a tessellation

The class template `DrawableTessellation` is a terminal branch of `SphericalTessellation` that adds some visualization and testing capabilities. The member function `DrawGridArcs()` outputs the coordinates along each of the curved edge of a tessellation at division `div` as a curve projected onto the plane of the screen. The projection is constrained by the rotation angles `rot_z` and `rot_x`. The parameter `opaque` tells the code that the sphere should be treated as an opaque object, so that the points on its far side are not drawn. Finally, if the parameter `smooth` is set to `false`, each edge is drawn as a straight line instead of a curve. The data is output to a file with the name pattern `mesh_base.div.out`, where `base` is the polyhedron base (`tetrahedron`, `hexahedron`, etc.), and `div` is the division. An example is `mesh_hexahedron_3.out`. Graphing can be performed with any XY visualizer such as Grace or Veusz. Figure 1 was produced by running `DrawGridArcs()` for division 0.

Table 1: Mesh element counts and the ratios between the largest and the smallest element at divisions 0–5 for all five basic solid shapes.

Base	division	verts	edges	faces	φ ratio	β ratio	Ω ratio
4	0	4	6	4	1.000	1.000	1.000
	1	10	24	16	1.644	2.667	3.000
	2	34	96	64	2.192	3.633	5.215
	3	130	384	256	2.452	3.906	6.180
	4	514	1536	1024	2.535	3.977	6.472
	5	2050	6144	4096	2.557	3.994	6.548
6	0	8	12	6	1.000	1.000	1.000
	1	26	48	24	1.276	1.333	1.000
	2	98	192	96	1.276	1.586	1.379
	3	386	768	384	1.276	1.753	1.600
	4	1538	3072	1536	1.281	1.861	1.730
	5	6146	12288	6144	1.281	1.926	1.809
8	0	6	12	8	1.000	1.000	1.000
	1	18	48	32	1.222	1.644	1.622
	2	66	192	128	1.491	1.904	1.962
	3	258	768	512	1.542	1.976	2.068
	4	1026	3072	2048	1.555	1.994	2.096
	5	4098	12288	8192	1.558	1.999	2.104
12	0	20	30	12	1.000	1.000	1.000
	1	32	90	60	1.119	1.200	1.000
	2	122	360	240	1.162	1.292	1.073
	3	482	1440	960	1.174	1.324	1.094
	4	1922	5760	3840	1.177	1.331	1.099
	5	7682	23040	15360	1.178	1.333	1.100
20	0	12	30	20	1.000	1.000	1.000
	1	42	120	80	1.135	1.235	1.203
	2	162	480	320	1.171	1.308	1.275
	3	642	1920	1280	1.191	1.327	1.294
	4	2562	7680	5120	1.194	1.332	1.299
	5	10242	30720	20480	1.195	1.333	1.300

The remaining functions of `DrawableTessellation` are used for debugging and testing. `PrintStats()` prints a table of geometric properties of the tessellation at every division between 0 and `max_division`. Table 1 gives the element counts and the mesh non-uniformity coefficients for each of the tessellation types. These include the disparities in face areas, vertex angles, and edge arc lengths. At division 0 all faces and edges are identical, but at subsequent divisions they are not because of the topological properties of the manifold (for example, the face in the center in Figure 2 has a smaller area than the three surrounding faces. A

good mesh must be as uniform as possible. From Table 1 one can immediately see that the tetrahedral mesh is exceptionally poor, so it is effectively useless. The hexahedral and octahedral meshes are better, but the former has the advantage of having quadrilateral faces which can significantly simplify many mesh operations; for this reason it is often used in research (as the so-called “cube sphere”). The dodecahedral and icosahedral meshes have the best properties and should be preferred over all others. The slightly better uniformity of the dodecahedral mesh is offset by the more complicated bookkeeping necessary to distinguish between pentagonal and triangular faces; for this reasons the icosahedral mesh is used more often.

The functions `TestConnectivity()` and `PrintConn()` perform a test of neighbor connectivity tables and print those tables, respectively; these are most helpful during debugging. `DrawableTessellation` is declared in `drawable_tessellation.hh`, and its functions are defined in `drawable_tessellation.cc`. There is no need to include any of these in a production code. The test code `main_test_drawable_tessellation.cc` is included in the `tests` subdirectory.

2.4 Helpful interface functions

The derived class `RequestableTessellation`, declared in `requestable_tessellation.hh` and implemented in `requestable_tessellation.cc`, adds the important `Locate()` function that finds the face containing a given point on the unit sphere. This function first calculates the scalar products of the point with every division 0 vertex (at most 20 for `POLY_DODECAHEDRON`, but most often either 12 or 8). The smallest value gives the vertex nearest to the point. Then the function searches among all the neighbor faces of the nearest vertex to find which of them contains the point. A test for the face interior involves computing scalar products of the point with every edge normal (a vector product of the vertices defining the edge); if all products are positive, the point lies in the interior of the face. The helper function `VectorInsideFace()` perform this verification. Finally, `Locate()` will recursively search among the children of the division 0 face to test which one of them contains the point. In addition to `Locate()`, `RequestableTessellation` provides functions to generate neighbor lists for a given face, vertex, or edge with names like `VertNeighborsOffFace()`, etc. These are later used to calculate the “exchange sites” (a collection of face, edge and vertex neighbors that can be used when exchanging information across sector boundaries, see the grid block class hierarchy description).

There are also protected functions to find the parent face of a given face, `GetParent()`, and to return a list of children of a given face, `GetChildren()`. These operations are very computationally inexpensive because face numbering follows a simple pattern. For example, the parent face index f_p of a face with an index f at division d is computed as

$$f_p = \frac{f - N_f^{d-1}}{N_c^{d-1} - 1}, \quad (7)$$

where N_c^{d-1} was defined above. The exception to this rule is the face that inherited the index of the parent, in which case the parent’s index is the same as that of the child. The helper function `GetMinDivision()` takes one argument (face index) and returns the minimum possible division of this face, which immediately tells us whether the child’s index was inherited. `RequestableTessellation` also provides a public function to return a list of faces in a sector in a tree format. This routine, named `GetAllInsideFaceTree()`, uses the recursive protected function `DescendTree()` to find all leaf nodes in a tree with a given root. Finally the public method `FillVertCoordArrays()` is provided to return a set of vertex coordinates based on a list of their indices. As will be shown in Section 3, this function plays an important role in setting up the geometry of each grid block.

2.5 Providing information to grid blocks

The final derived class in this hierarchy is called `TraversableTessellation`. This class, declared in the header file `traversable_tessellation.hh` and implemented in `traversable_tessellation.cc`, adds a very important (and very complex) function `GetAllInsideFaceNative()`. This is a “sector crawler”, whose role is to visit every face that belongs to a sector, including its halo of ghost faces, and record every face and vertex encountered along the way. The input argument include the sector division `divf`, the face division `divf`, the sector index `sect`, and the thickness of the ghost halo, while the function returns the status of the corners (singular or not) and the list of all faces and vertices that belong to the sector plus halo. The crawler only knows how to perform “steps”, codified in the protected routine `Step()`. There are three types

of steps for triangular meshes and four for quad meshes as shown in Figure 5. For example, a Type 1 step on a triangular mesh moves from the starting face to its S neighbor, while the base vertex moves to its E neighbor, etc. While there are only three step types defined for triangular meshes, in reality there are 6, because the starting face can have two possible orientations. However, the code automatically rotates the “view” 180° after each step, and the crawler is unaware it is in fact going in the opposite direction.

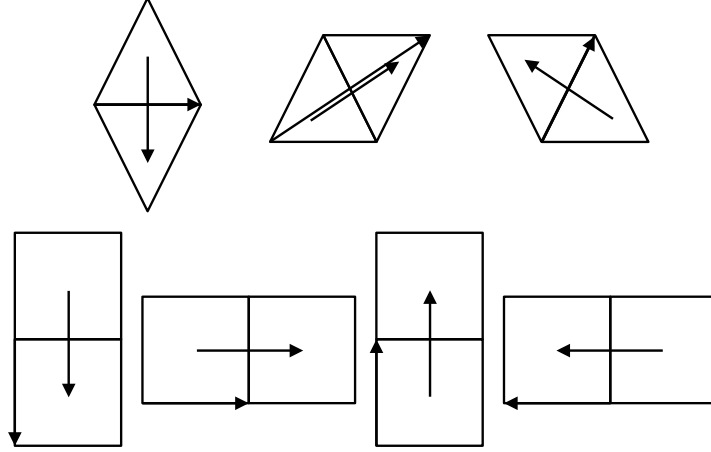


Figure 5: Steps performed by the sector crawler function for triangular (top) and quadrilateral (bottom) meshes. One of the arrows shows the face move and the other the base vertex move.

The sector crawler starts at the base vertex of the sector. It then takes a sequence of steps to locate the SW corner of the sector halo. From that position the face traversal routine will record every face and vertex encountered in the arrays `flist` and `vlist` respectively (the arrays are provided by the caller and must have the sufficient storage capacity). The full procedure consists of three sub-tasks where the crawler traverses one diagonal or vertical band of faces (a diagram is provided in the comments). The reason for this is that the list of faces/vertices must be in a specific order prescribed by the Triangular Addressing System (TAS) or Quadrilateral Addressing System (QAS) used by grid blocks (see Section 3). At the end, a complete list of vertices and faces is returned to the calling program. Because of the complexity of this routine, it should only be modified by the framework developers.

2.6 Tessellation class hierarchy

Figure 6 shows the relationship between the classes dealing with spherical tessellations. A production code should only include the header `traversable_tessellation.hh`. The common use in a parallel environment is to define one instance of the class in each worker process. SPECTRUM uses a computational unit known as a *grid block*. In the OOP approach each grid block only needs to know its own location in the mesh and some basic information about its surroundings, and all such information is provided by a tessellation object. In principle, a client-server model can be used where a separate boss process services requests from the worker processes, but it would be extremely inefficient for location requests involving very short and very frequent messages, so a local instance is preferred. The public method `PrintStats()` of `DrawableTessellation` can be used to figure out the memory footprint, if it is a concern.

3 Grid block as computational units

The tessellation framework described above has a number of limitations. It is monolithic, in the sense that a single object describes the entire mesh on a sphere. It is also restricted to surface modeling and does not have any provision for the third (radial) dimension. To efficiently divide the computational load between

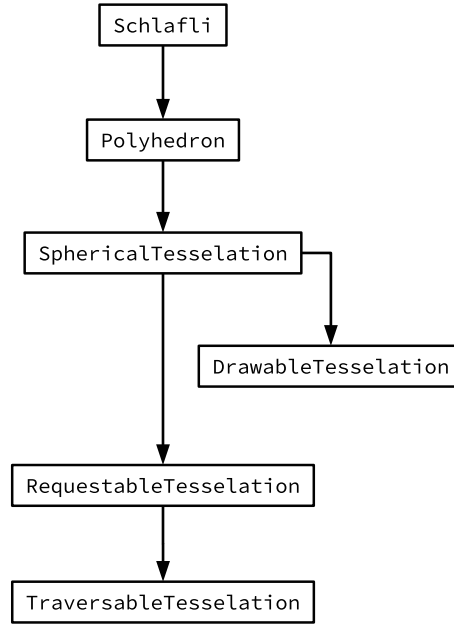


Figure 6: The tessellation class hierarchy in SPECTRUM.

multiple processes requires a smaller mesh unit comprised of only a handful of computational zones. In the literature such a unit is typically called a block.

A *grid block* is a computational unit in the fluid simulation framework of SPECTRUM. A block cannot be split between processes, but a process can have more than one grid block assigned to it. A grid block has the shape of a *frustum*, a type of prism; it is a tensor product of a geodesic *sector* on a sphere and a concentric extrusion in the radial direction called a *slab*. In theory, the smallest possible grid block is just one face wide by one shell thick. On the other end, while in principle, the entire simulation can be thought of as one large grid block, this is not currently implemented in SPECTRUM; the widest grid block spans a single division 0 face (but can contain any number of spherical shells). Therefore, the smallest number of blocks in a simulation is equal to the number of faces of the base polyhedron (4, 6, 8, 12, or 20).

A grid block is subdivided into computational *zones* or cells that are the smallest volume elements in the mesh. Figure 7 shows an example of a zone in a triangular mesh in the shape of a truncated triangular pyramid (frustum). SPECTRUM can also use quadrilateral zones based on a hexahedral tessellation. A zone has two t-faces (spherical polygons) as its top and bottom “caps”, 3 or 4 r-faces (annular trapezoids) as the sides, 6 or 8 t-edges (curved), 3 or 4 r-edges (straight), and 6 or 8 vertices. When referring to sectors, it will be said that they consist of t-faces (or just faces), while slabs consist of r-shells, or simply shells.

Unlike the tessellation, grid blocks can be freely duplicated and copied. This is in fact required because the simulation level object (see below) will contain an `std::vector` of grid blocks which performs a copy whenever the vector is resized, so each base and derived class in this family is provided with copy and move constructors. Assignment operators are not implemented at this time because direct assignment is rarely if ever needed. Another peculiarity of the grid block family of classes is that their memory allocation is a multi-stage process. The functions that correspond to different stages of the setup take an extra argument, `construct` that should be set to `false` in the user code. This tells the code that the function is not called from within a constructor, and therefore it can safely call the parent version. If, on the other hand, the function is called from within a copy constructor, the argument must be set to `true` to inform the code that the preceding stages were already executed by the constructor of the base class.

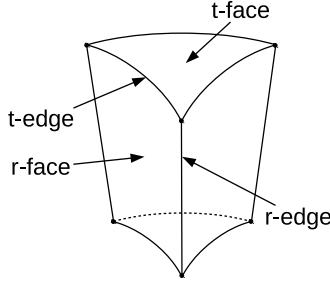


Figure 7: Geometric elements of a computational zone.

3.1 Radial extrusion

As with tessellation objects, all C++ classes implementing grid blocks can be found in header and source files residing in the `geodesic` sub-directory of the SPECTRUM source. The radial grid numbering is implemented as a very simple class `SphericalSlab` which is fully described in the header file `spherical_slab.hh`. The class defines the number of interior shells H (`n_shells`) and the number of ghost shells on the top and the bottom of the slab (`ghost_height`) and provides a public function `IsInteriorShell()` to test whether a shell with a given index is located in the interior of the slab. The function `SetDimensions()` is used to specify the number of elements in the slab. The arguments are `height`, which is the number of interior shells H , and `hghost`, which is the number of ghost shells above and below the interior (G , the notation also used below for the number of ghost faces in a sector; both are typically set to 2). The number of interior shells H is stored in the protected variable `n_shells`, G is stored in `ghost_height`, and the total height of the slab $H + 2G$ in `n_shells_withghost`. Note that `SetDimensions()` is a key function of the grid block family; it is used in all derived classes to allocate dynamic memory for the object.

3.2 Sector and face orientation conventions

The implementation of the geodesic mesh is far more complex. The code uses the so-called *polygonal addressing* scheme to locate an element in the sector using a pair of indices (i, j) , where “polygonal” can be either triangular or quadrilateral. At the foundation level of the grid block class hierarchy is the `PolygonalAddressing` class template implemented in the header file `polygonal_addressing.hh`. It takes one template parameter, an integer `verts_per_face` that can be either 3 or 4. This parameter describes the shape of the sector (triangular or quadrilateral). Pentagonal sectors, while in theory possible with a dodecahedral tessellation, are not implemented in SPECTRUM. If the number of vertices per face is p , then the number of edges that meet at a vertex q can be found as

$$q = \frac{2p}{p-2}, \quad (8)$$

which evaluates to either 6 (for $p = 3$, TAS) or 4 (for $p = 4$, QAS). Tiling with hexagons ($p = 6$, $q = 3$) is possible in theory, but not implemented algorithmically due to a lack of nesting property (unlike triangles or quads, a hexagon cannot be sub-divided into smaller hexagons without an overlap). Still, hexagonal geodesic meshes are often used in atmospheric modeling and motivated computer science students are welcome to add this functionality to the code.

To use SPECTRUM effectively it is essential to develop a solid spatial understanding of the sector shapes and zone orientations. A sector is a curved surface that can be oriented in a multitude of different ways in the global three-dimensional mesh. Moreover, the zones that comprise the sector are not identical in shape and size, for the reasons explained in Section 2. Here we will introduce a simplified conceptual representation of a sector as a flat equilateral polygon divided into equal parts. To fix the viewpoint, the polygon will

be oriented in the plane of the page such that at least one side is horizontal. For a triangular mesh the horizontal side is at the lowest point and the opposite vertex is at the highest point in the image, while in a quad mesh there are two horizontal sides. In a triangular mesh faces can be oriented either up (as the sector itself) or down. The upward orientation is considered to be the default. A downward face is a mirror image of an upward face, and all operations on the former are rotated by 180° compared to the latter. The quad mesh is free from such complexities because a square has vertical symmetry.

The `PolygonalAddressing` class implemented in the header file `polygonal.addressing.hh` defines the *relative* positions of the elements and establishes the principles governing mesh connectivity. The class provides functions to compute the number of elements inside a triangle or square with side length l , N_v , N_e , and N_f referring to vertices, edges, and faces, respectively. They are all named according to the same pattern (e.g., `EdgeCount()`). The expressions are

$$N_e = \frac{ql(l+1)}{2S}, \quad (9)$$

$$N_f = l^2, \quad (10)$$

$$N_v = N_e - N_f + 1 \quad (11)$$

(the last one can be recognized to be the Euler formula). The number S is stored in the constant `square_fill` that is a protected member of `PolygonalAddressing`; it refers to the number of polygons to fill a square (2 for triangles and 1 for quads). The constant `cardinal_directions` is one half of q ; it gives the number of different edge orientations in the mesh. In a quad mesh there are two types of edges (horizontal and vertical), while in a triangular mesh there are three types (one horizontal and two diagonal).

Several protected data member arrays are initialized in `PolygonalAddressing` that assist in finding the indices of neighbor elements given the index of an element of a different type. For example, the array `vert_face` stores the values that must be added to the (i, j) vertex coordinates to obtain the coordinates of that vertex's face neighbors. Basically, these are “bootstrap” data structures used only in the generation of the connectivity arrays in the derived classes (that have much broader uses). In general, `PolygonalAddressing` contains only low-level technical information and should not be instantiated explicitly in a user code. However, the comments in the `Setup()` routine that is called from the constructor to fill in the bootstrap arrays contains useful diagrams showing neighbor relationships between the elements that can be of help to developers.

3.3 Element relationship within a sector

The `GeodesicSector` class template is derived from `PolygonalAddressing` and has the same template argument. The class interface can be found in the header file `geodesic_sector.hh` and its implementation in `geodesic_sector.cc`. Unlike its base class, which deals with relative displacements only, a sector fixes the side length of the sector and the element counts either via a constructor or the public function `SetDimensions()`. The arguments to `SetDimensions()` are the length of the side of the sector's interior L (`width`), stored in the member variable `side_length`, and the number of the ghost faces G (`wghost`), stored in `ghost_width`. Note that there is yet no concept of physical coordinates; the mesh elements are purely abstract concepts within this class.

Some important protected variables of `GeodesicSector` include element counts (`n_verts`, `n_edges` and `n_faces` for the interior part, and `n_verts_withghost`, etc. for the sector plus the ghost halo), the arrays mapping the (i, j) coordinates into the element indices (`vert_index_sector`, `edge_index_sector`, `face_index_sector`), the inverse lookup arrays used to obtain a (i, j) pair based on the index (`vert_index_i`, `vert_index_j`, `edge_index_i`, etc.), the *local* connectivity arrays `vv_local`, `ve_local`, `vf_local`, `ev_local`, etc.) and the binary masks `vert_masks`, `edge_masks`, and `face_masks` whose roles are to mark the elements as having certain properties, such as being in the interior of the sector or subject to reconstruction (see below). Unlike in the tessellation, the rules governing element connectivity in a sector are quite simple. However, there are exceptions to these rules near the vertices of the base polyhedron, also known as “singular corners”. Such exceptions are implemented in derived classes (see below).

Figure 8 shows the numbering scheme for faces in the triangular (TAS) and quadrilateral (QAS) addressing systems. When discussing these systems we always orient the sector such that its *base vertex* (the first entry in the `fv_local` list) is on the lower left (or SW). In TAS diagrams the upward faces have white

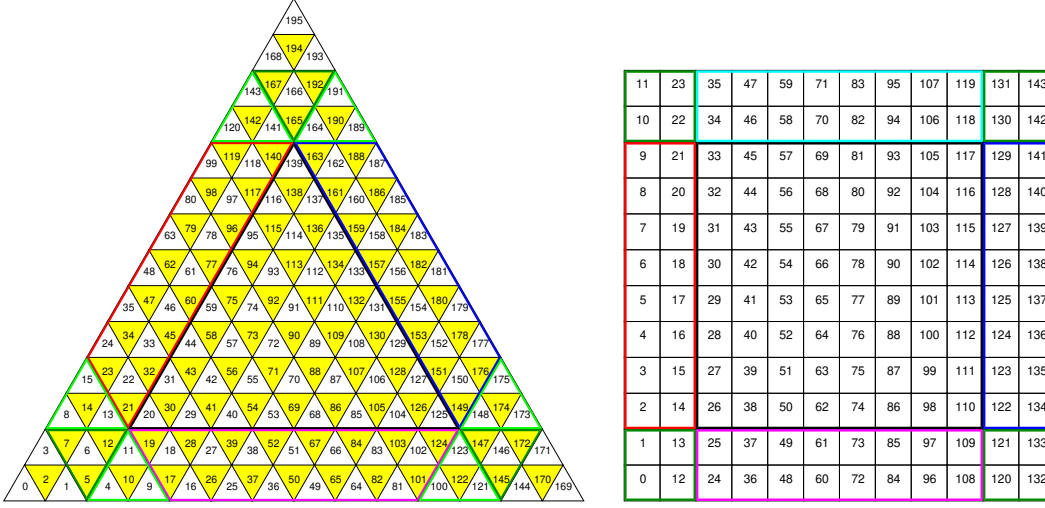


Figure 8: Face numbering in the TAS (left) and QAS (right). Both sectors have $L = 8$ and $G = 2$.

background and the downward faces are yellow; in the QAS there is no distinction between the two, so all faces have the white background.

The i (“horizontal”) face index runs from 0 to `total_length`, the total length of the side, including ghost faces. Its value, L_t , is $L + 3G$ in the TAS and $L + 2G$ in the QAS, where G is the number of ghost faces on each side. Using the constant `square_fill` defined above we can express these with a single equation

$$L_t = L + (1 + S)G. \quad (12)$$

While in principle grid blocks can use sectors of any size, in practice L will be a power of 2 because the mesh generation uses the recursive bisection algorithm described in the tessellation documentation. In that case, if the sector division is d_s , and the face division is d , then

$$L = 2^{d-d_s}. \quad (13)$$

It is recommended to have $d - d_s \geq 3$, so that L is at least 8 for $G = 2$. Large sectors are more efficient to process than small sectors because they have a larger ratio of the number of interior faces to the total number of faces. For example, using $G = 2$ and TAS, eq. (10) yields the ratio of 0.33 for $L = 8$, 0.53 for $L = 16$, and 0.71 for $L = 32$.

Each element in the sector (vertex, edge, or face) can be addressed in two ways. The first is directly by its index, which runs from zero to the number of elements of the appropriate type minus one. The second is through a pair of indices i and j . The former is measured horizontally from left to right in both TAS and QAS, but the j (“diagonal” or “vertical”) index is measured differently in the two systems. In the QAS it runs upward from 0 to `total_length`, just like the i index. However, in the TAS it runs diagonally from the lower right to the upper left (SE to NW), starting from 0, and its largest value is equal to $2i$ for faces and i for edges and vertices.

The edge numbering scheme is shown in Figure 9, and the vertex numbering scheme is shown in Figure 10. Note that to visit all faces and vertices in a mesh, two nested loops (in i and j) are sufficient, but for the edges an extra loop on the edge type (0–2 for TAS and 0–1 for QAS) must be added.

The public interface of `GeodesicSector` contains default, copy, and move constructors and a destructor, the above-mentioned `SetDimensions()`, and the memory de-allocator `FreeStorage()`. A few trivial helper functions return the number of elements (e.g., `InteriorFaces()`, `InteriorVerts()`, etc.). There are also

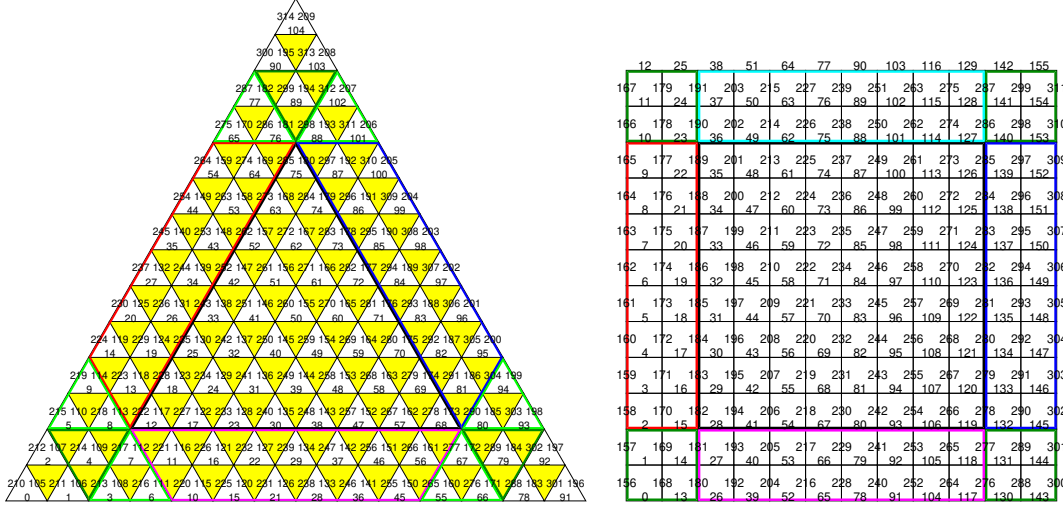


Figure 9: Edge numbering in the TAS (left) and QAS (right). Both sectors have $L = 8$ and $G = 2$.

debugging functions `PrintAddresses()`, `PrintConn()`, and `PrintMask()` whose functions are obvious from the name. They all take one argument, the type of information to be printed. For example, passing a value of 1 to `PrintAddresses()` will print a table of (i, j, v) values, where v is the vertex index corresponding to the (i, j) pair. Protected member functions include `ComputeIndices()` that computes forward maps such as `vert_index_sector`, `face_index_sector`, etc., and the inverse maps stored in `vert_index_i`, `vert_index_j`, `edge_index_i`, etc. There are also 8 connectivity evaluators such as `VertVertConn()`, `VertEdgeConn()`, etc., that compute the neighbor relationship between the elements in a sector. These functions are also responsible for setting the mask values of `GEOELM_NEXI` for non-existing elements, `GEOELM_INTR` for interior elements, `GEOELM_BNDR` for elements on the boundary (does not apply to faces), and `GEOELM_GHST` for elements within the ghost halo. A unique feature of the TAS is that the triangular sub-blocks near sector vertices (those that have no color border in Figures 8, 9, and 10), are considered non-existent.

A branch derived class `DrawableSector` is provided for visualization purposes to better understand the relationship between the elements in the mesh and between neighboring sectors. This class, residing in `drawable_sector.hh` and `drawable_sector.cc`, provides a single public function `Draw()` which outputs a PostScript file with a vector drawing of a sector with element indices printed next to them and special regions (see below) shown with different border colors. Figures 8, 9, and 10 are all examples of its output. This class is intended to be used mainly as an educational or debugging tool; it should not be included in a production code. A simple test code `main_test_drawable_sector.hh` is included in the distribution. By default it calls the `Draw()` function for triangular and quadrilateral sectors with the a length of 8 and a ghost width of 2. The output is saved in the files `face_3.8.2.eps`, `edge_3.8.2.eps`, `vert_3.8.2.eps` for TAS and `face_4.8.2.eps`, etc. for QAS. The code can be readily modified to print a sector of any desired side length and number of ghost faces.

3.4 Blocks with physical dimensions

The `GridBlock` class template, declared in `grid_block.hh` and implemented in `grid_block.cc`, is derived from both `SphericalSlab` and `GeodesicSector`. It represents a real physical volume of space and stores the Cartesian coordinates of every vertex of the sector on the unit sphere (`block_vert_cart`) and the radial shell spacing. The latter is typically non-uniform, starting with fine spacing close to the origin that becomes progressively more coarse with radial distance. To simplify the mapping process, the code

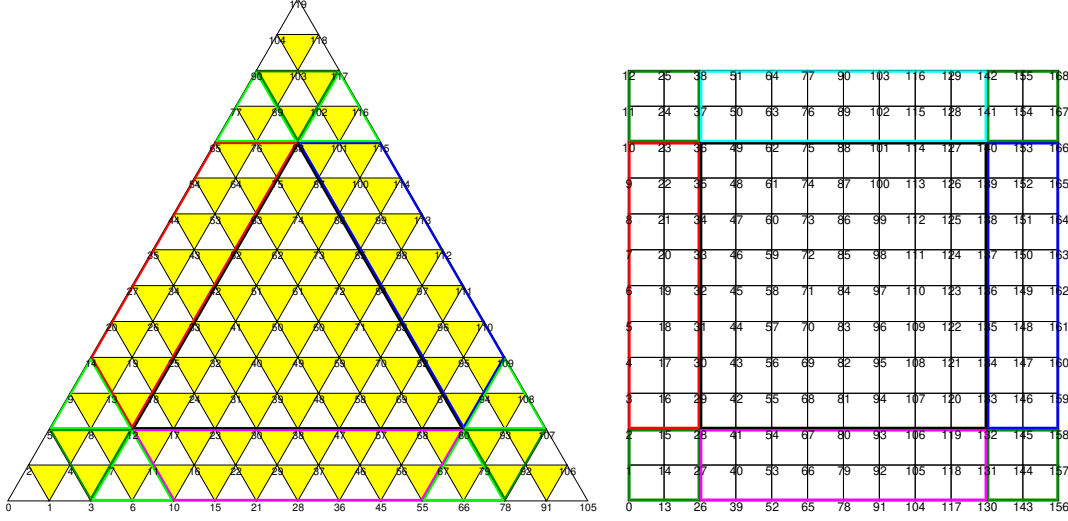


Figure 10: Vertex numbering in the TAS (left) and QAS (right). Both sectors have $L = 8$ and $G = 2$.

introduces a pseudo-uniform reference or “index” variable ξ that always varies between 0 and 1 and whose spacing $\Delta\xi$ (data member `dx_i`) is uniform, unlike the variable Δr that is represented by the array `dr`. The mapping is performed with the help of the member variable `dist_map`, which is a `std::shared_ptr` to an object of polymorphic type `DistanceBase`, defined in `distance_map.hh` and `distance_map.cc`. That class, derived from `Params`, provides three mapping functions, `GetPhysical()` returning the $\xi \rightarrow r$ map, `GetReference()` returning the inverse map $r \rightarrow \xi$, and `GetDerivative()` that returns the derivative $\xi \rightarrow dr/d\xi$. All evaluations are performed internally via a protected method `EvaluateDistance()`. A description of distance maps available in SPECTRUM can be found in Appendix B.

As we shall see below, it is more efficient to perform variable reconstruction using the reference radial distance rather than physical distances because it allows factoring the variable reconstruction process into operations performed in the radial direction and those on the sphere which reduces the storage requirements and provides performance advantages. The distance mapping classes themselves are not geodesic mesh specific and they reside in the directory `geometry` of the SPECTRUM source tree.

Initializing a grid block is a four-stage process. The first stage is performed in `Setup()` that computes the *rotation matrices*. These matrices allow translating the TAS/QAS indices of an arbitrary sub-block whose addressing system is rotated relative to the block coordinates, into the global TAS/QAS indices of the block. For vertex translation one uses the three-dimensional array `rotated_verts`. The first index refers to the rotation amount, the second index selects the i or j coordinate in the global block frame, and the third selects the i or j coordinate in the rotated sub-block. If the sub-block has a base vertex with coordinates `i0`, `j0`, and a vertex has coordinates `ir`, `jr` in the sub-block frame rotated by the amount `rot`, then the vertex’s global coordinates are computed as

```
i = i0 + ir * rotated_verts[rot][0][0] + jr * rotated_verts[rot][0][1];
j = j0 + ir * rotated_verts[rot][1][0] + jr * rotated_verts[rot][1][1];
```

Addressing rotated edges is similar to rotated vertices, except that an additional shift array is required (`rotated_shift`). Its value is added to the expressions for the vertices given above. Face translation uses a slightly different format to accommodate both upward and downward faces of TAS. The array `rotated_faces` used for TAS has separate entries for odd and even `jr`. The translation is done by the following code fragment:

```
i = i0 + ir * rotated_faces[rot][0][0] + jr * rotated_faces[rot][0][1 + jr % square_fill];
j = j0 + ir * rotated_faces[rot][1][0] + jr * rotated_faces[rot][1][1 + jr % square_fill];
```

Here `i0` and `j0` are the global coordinates of the face that owns the base vertex. `Setup()` is always called from the constructor and need not be invoked explicitly.

The second stage of the initialization process is performed in `SetDimensions()`, which calls the versions of both parent classes and does additional storage allocation. This step could be called from the constructor, so it has an extra boolean argument `construct` that is set to `true` if that is the case, and to `false` if it is invoked from the derived class's overloaded method. Next, each grid block must be assigned a unique index, stored in `block_index` (typically it is simply a product of the respective sector and slab indices). The index must be set by calling `SetIndex()` and can be retrieved with `GetIndex()`.

The fourth, most complicated step, is performed in the public function `AssociateMesh()` that imports the unit-sphere vertex coordinates and the radial shell spacing as provided by the controlling code. It also informs the block whether there is a neighbor above and below (stored in a boolean array `border_type`) and gives the status of the corners (stored in `corner_type`). When the value stored is false, the block is not at a physical radial boundary, and the corresponding corner is not singular (i.e., has a full compliment of neighbor blocks, either 5 or 3). A value of true stored in `border_type` indicates that a block is located in either the first or the last slab (or both), which tells the code that some physical boundary condition must be applied there. A value of true in `corner_type` is an indication that the corner is singular, i.e., it is a vertex of the base polyhedron, and the number of neighbors is reduced by one to 4 and 2, respectively. Furthermore, the mesh has defects at these corners. For example, in Figure 8, the dark green triangles at the corners could be missing; in their place a *cut line* appears that must be closed by folding the figure out of the plane of the page. The connectivity rules must be modified near the resulting seam. For example, neighbor 2 or face 4 would be face 8 instead of face 5. The number of singular corners `n_singular` can be zero, one, or equal to `verts_per_face`, no other values are physically possible.

`AssociateMesh()` takes 7 arguments: `ximin` and `ximax` are the values of the reference variable ξ at the bottom and the top of the block respectively (not counting the ghost shells), `corners` tells the class which, if any, of the block's corners are singular, `borders` tells it that the top or the bottom of the block is at the simulation boundary, `vcart` is the array of vertex coordinates, and `dist_map_in` is the object (passed as a `std::shared_ptr`) describing the radial mesh spacing. The last argument, `construct`, has the same effects as for `SetDimensions()`. The arrays `vcart` and `corners` can be obtained by calling `GetAllInsideFaceNative()` followed by `FillVertCoordArrays()`, both of which are methods of the `TraversableTessellation` class. The following code fragment illustrates the grid block setup sequence (taken almost without change from the test code `main_test_grid_block.cc`)

```
#define POLY_TYPE POLY_ICOSAHEDRON
const int verts_per_face = 3;

const double rmin = 1.0;
const double rmax = 100.0;
const double power_law = 2.0;

const int face_division = 4;
const int sect_division = 1;
const int ghost_width = 2;

const int slab_height = 8;
const int n_slabs = 30;
const int ghost_height = 2;
const int sector = 1;
const int slab = 1;
const int bidx = slab * sector;

bool corners[3];
bool borders[2] = {false, false};

// Create a Tesselation object
```

```

TraversableTessellation<POLY_TYPE, face_division> tess;

// Set up a radial map
DataContainer container;
container.Clear();
container.Insert(rmin);
container.Insert(rmax);
container.Insert(power_law);

std::shared_ptr<DistanceBase> dist_map = std::make_shared<DistancePowerLaw>();
dist_map->SetupObject(container);

// Create one grid block
GridBlock<verts_per_face> block(Pow2(face_division - sect_division), ghost_width,
                                slab_height, ghost_height);

block.SetIndex(bidx);

int *face_index = new int[block.TotalFaces()];
int *vert_index = new int[block.TotalVerts()];
GeoVector* vcart = new GeoVector[block.TotalVerts()];

tess.GetAllInsideFaceNative(sect_division, sector, face_division, ghost_width,
                             face_index, vert_index, corners);
tess.FillVertCoordArrays(block.TotalVerts(), vert_index, vcart);
block.AssociateMesh(slab / (double)n_slabs, (slab + 1.0) / (double)n_slabs,
                    corners, borders, vcart, dist_map, false);

```

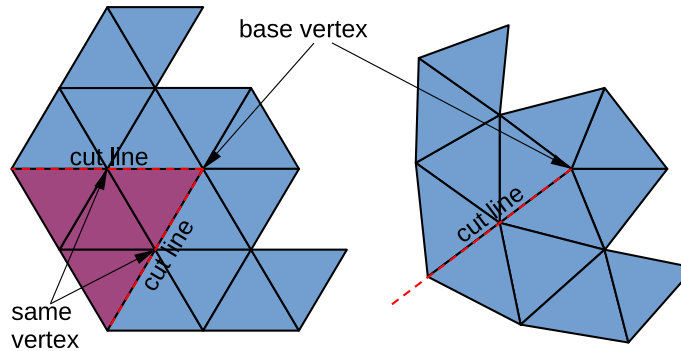


Figure 11: A triangular sector with a singular cornered before (left) and after (right) the gap between the cut lines (red dashed lines) was closed. The purple colored faces do not exist in the mesh. Note also two sets of vertices (and edges) on each side of the cut line that present a duplication problem.

One of the tasks that `AssociateMesh()` performs is calling the protected method `FixSingularCorners()` which “repairs” the connectivity tables near the seam of each singular corner. The procedure is illustrated in Figure 11. The code makes two passes for each singular corner. In the first pass it locates the elements that do not exist and flags them. It resets the connectivity tables of the non-existing elements inside the gap. For the vertices and edges on the cut line it raises the mask bit `GEOELM_CUTL`. Because there are two sets of vertices and edges on the cut line, one of them must be eliminated; those are stored in protected arrays `dup_vert` and `dup_edge`. In the second pass the code fixes the neighbor connections, such as `vv_local`, `ff_local`, etc. across the cut line. Some of this work seems redundant, but the results will be needed during flux calculation and neighbor block exchange operations. We remind the reader that in TAS the “tips” of

the sector (white regions without a color border in Figures 8, etc.) are never included in the face list, i.e., are considered non-existing and have the mask bit `GEOELM_NEXI` raised, regardless of whether the corner is singular or not.

Grid blocks are essentially 3D concepts. They operate primarily on zones (volume elements in the shape of a frustum), which are addressed via a shell index l and a face index f . In some situations we will have to deal with faces, edges, and even vertices. In what follows we will use e for edge indices and v for vertex indices in the TAS or QAS. We will use half indices ($l - 1/2$, $l + 1/2$, etc.) to measure the radial distances to the shell interfaces. Each shell is then defined by two numbers, the midpoint r_l and the width Δ_l

$$r_l = \frac{r_{l-1/2} + r_{l+1/2}}{2}, \quad \Delta_l = r_{l+1/2} - r_{l-1/2}. \quad (14)$$

Table 2 shows the addressing rules for the elements in the 3D mesh.

Table 2: Addressing of the elements in a grid block

	zone	t-face	r-face	t-edge	r-edge	vertex
First index	l	$l \pm 1/2$	l	$l \pm 1/2$	l	$l \pm 1/2$
Second index	f	f	e	e	v	v

3.5 Saving and visualizing grid information

SPECTRUM uses the SILO file format (<https://github.com/LLNL/Silo>) to store and retrieve the information about the mesh and the variables defined on the mesh. SILO stores the data in a platform-independent binary format and can be used for visualization with the VisIt software (<https://visit-dav.github.io/visit-website>). If saving the results is not required (e.g., during testing of one particular functional module), the code can be compiled and executed without SILO. In all other cases the pre-processor variable `USE_SILO` should be defined. In that case `GridBlock` adds two more functions, `GenerateSiloIndexing()` and `WriteSiloMesh()`. The first of these generates a lookup table mapping the QAS/TAS indices into SILO indices. Each block is written to a SILO database separately; the user can visualize either a single block or the entire simulation (i.e., every block in it).

To combine the blocks into a single object that can be viewed and manipulated as a whole, the *assembly* metadata must be provided. SILO does not require connectivity information between the blocks, only their names. However, to properly interpolate the values near block boundaries, VisIt requires *one* layer of ghost zones at each boundary. Simulations typically uses 2 or more layers, so the simulation zone indices do not match the SILO indices. A mapping table is computed by `GenerateSiloIndexing()` to facilitate index translations in all I/O operations. The function makes use of the protected function `IsInteriorVertOfSector()` to verify that the vertex belongs to the block interior. In addition to all internal vertices, all vertices that belong to faces that have at least one vertex on the sector boundary are included in the map. Care is required at singular corners to prevent duplicate inclusion of vertices lying on the cut line.

The public function `WriteSilo()` will write out the mesh data to a SILO database that is typically opened by the simulation-level routine and passed as the argument `silofile`. The second argument, `phys.units`, tells the code whether to convert the code units to physical units. Because there are no simulation variables associated with the block yet, the only information that this class is capable of storing is the mesh itself. It is output by `WriteSiloMesh()`, a protected member function. Following the standard SILO workflow, the routine first generates a vertex coordinate list as a 2D array. Then a *zone list* is generated, where each zone is described as an individual 3D shape, `DB_ZONETYPE_PRISM` for a triangular and `DB_ZONETYPE_HEX` for a quad mesh. Ghost zones must be stored *after* the interior zones, so the code writes in two separate places in the zone list array. Finally, an external face list is generated and an unstructured mesh object is created by a call to a SILO interface function `DBPutUcdmesh()`. Figure 12 shows how the mesh looks like in VisIt, showing all internal zones. The figure was produced by importing the file `onegridlock.silo`, produced by running the test code `main.test.grid.block.cc`, into VisIt.

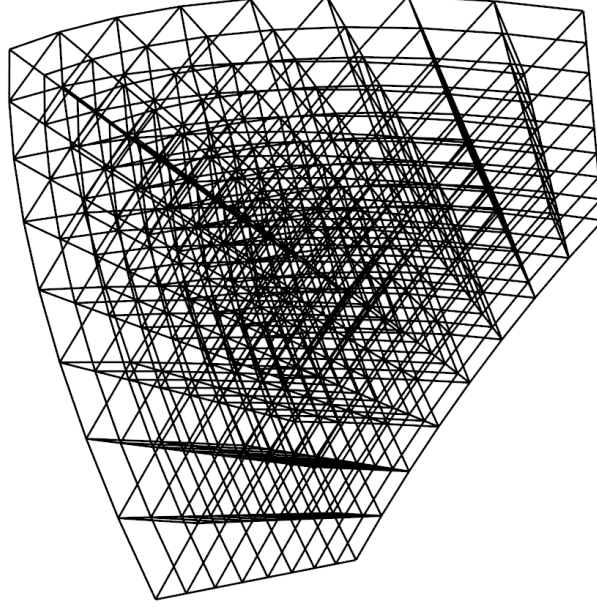


Figure 12: A visualization of a grid block (mesh only) with VisIt.

3.6 Preparing for reconstruction on the geodesic mesh

One of the responsibilities of a grid block is a generation of a sub-grid functional representation of the physical variables, a process known as *reconstruction*. The procedure is discussed in detail in Section 4. Here we will only describe the derived class `StenciledBlock` that adds geometrical objects to facilitate volumetric reconstruction of a zone variable, while the variables themselves are not defined yet.

Reconstruction involves multiple zones that together form a *stencil*. A stencil is defined for most zones in the block with the exception of those near the edges of the block. During a loop on zones subject to reconstruction, we will call the zone where reconstruction is performed (the loop index) the *principal* zone, while the remaining zones in the stencil will be called *stencil* zones. The current implementation uses second-order reconstruction, which means that each zone-centered conserved variable U can be represented as a degree one polynomial with 4 coefficients,

$$U_{lf}(\mathbf{r}) = \bar{U}_{lf} + \delta\mathbf{U}_{lf} \cdot (\mathbf{r} - \tilde{r}_l \mathbf{c}_f), \quad (15)$$

where l is the shell index, f is the face index, and $\mathbf{c}_f$ is the position of the center of mass of t-face f on the unit sphere (see Appendix A), which is implemented as the function `SphTriCenter()` declared in the header file `common/vectors.hh`. The scaling variable $\tilde{r}_l$, given by eq. (69) of Appendix C, converts the zone center of mass distance from the unit sphere to the actual 3D mesh. For quad faces the area and mass center are found by partitioning the quad into two triangles and adding their moments together. Be aware that $\mathbf{c}_f$ itself does not lie on the unit sphere.

The zone average quantity $\bar{U}_{lf}$ is always known, so only the three first-order modes $\delta\mathbf{U}_{lf}$, must be determined. Because radial and angular coordinates are independent on a concentric grid, the 3D representation of a variable (15) can be *factored* into a 1D by 2D representation, where 1D refers to the radial dependence and 2D refers to the variation on the unit sphere. The procedure can be further optimized if the radial shell spacing is exponentially rationed because in this case each zone with the same value of f has the same aspect ratio regardless of the shell index. Below we will show how to modify the reconstruction procedure so that *any* radial mesh mapping can be used with the same benefits.

The coefficients $\delta \mathbf{U}_{lf}$ are found from the requirement that the averages of the functional representations over the zones adjacent to the principal should be equal to the actual values of the variable in those zones. The zones selected for this matching comprise the reconstruction stencil of the zone (l, f) . The number of zones in the stencil must be equal to or greater than the number of degrees of freedom (DOFs) in the reconstruction. When the two are equal, the resulting reconstruction might not be very smooth if the constraints are too severe (this could happen, for example, near a discontinuity in the solution). For this reason it is recommended to use larger stencils, in which case the constraints can be satisfied in the least squares sense. Another technique is to use multiple stencils. One of the stencils (called central) is symmetric about the reconstructed zone, while the others are directional (skewed to one side). SPECTRUM uses 6 and 8 directional stencils for the triangular and quad meshes, respectively. Figure 13 shows the central and one directional stencil for one particular zone of the triangular mesh. Note that the principal zone itself ($l = 3, f = 57$) was not included in the stencil and was omitted from the drawings.

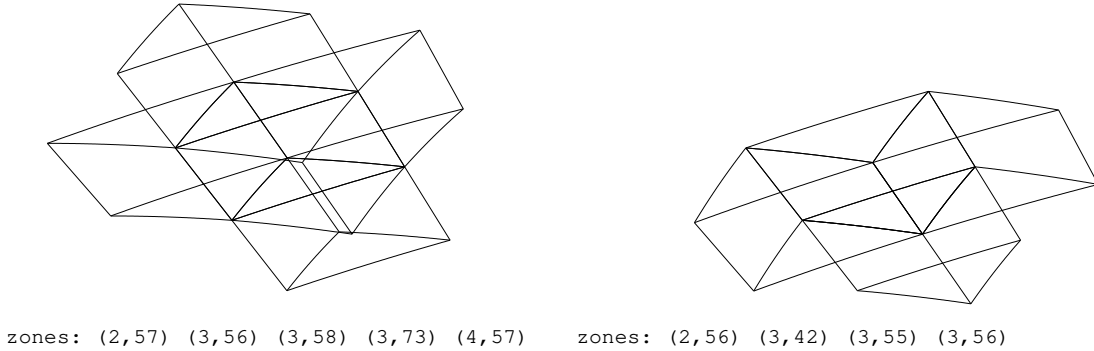


Figure 13: A central stencil (left) and one directional stencil (right) for the face (3,57) in a triangular mesh.

The setup sequence of the **StenciledBlock** includes, in addition to the base class operations, compiling a list of faces for which reconstruction will be performed. This includes all interior faces plus one layer of ghost zones. The protected method **MarkStenciledArea()** sets the bit **GEOELM_STEN** in the face mask for such faces. All stencils are constructed in the function **BuildAllStencils()**. Each stencil has a record in the three-dimensional array **stencil_zonelist**. The first index is the index of the principal face. The second index corresponds to the stencil type (0 is central, and the rest are directional). The third index numbers the zones in the stencil. There are two entries per zone: the face index and the shell *displacement* that follow one another. As mentioned earlier, the zone list does not include the principal zone. A central stencil includes r-face neighbors (3 or 4) and the zones above and below the principal. A directional stencil includes one r-face neighbor and two of the latter's r-face neighbors nearest to the principal, plus one zone either above or below the neighbor (Figure 13).

In the third step in the setup sequence the code computes the t-edge lengths, t-face areas and the centers of mass of the corresponding circular arcs and spherical triangles by calling **ComputeMoments()**. Mathematical expressions for these operations may be found in Appendix A. Finally, the last step computes the matrices that will be used in the reconstruction process. These matrices depend only the geometric properties of the zones and are described in detail in Section 4. There is only one matrix per face per stencil regardless of the number of shells in the block. The code computes the LU decomposition, which is equivalent to finding the inverse. All matrix operations are performed with the help of the Eigen3 library (https://eigen.tuxfamily.org/index.php?title=Main_Page). The stenciled block class also has two debug functions, **PrintStencilProps()** and **DrawStencil()**. The latter was used to generate Figure 13. The interface should be familiar by now; the arguments are the shell **shell**, the principal face index **pfac**, the stencil (self-explanatory) and the rotation angles for the plane projection. The function simply calls **GridBlock::DrawFace()** for each face in the stencil.

3.7 Dealing with neighbors

Grid blocks exist within the global mesh, and at times they require information from neighboring blocks. This is most often to perform reconstruction in zones lying on or near the boundary because their stencils protrude into the ghost region. Blocks never communicate with other blocks directly; they contact the intermediaries whenever external information is required. The next class in the grid block hierarchy, named `BufferedBlock`, add the capability to communicate with such intermediaries.

The file naming follows the now familiar pattern (`buffered_block.hh`, `buffered_block.cc`). The class has storage allocation (`cons_vars`) for the conserved variables of type `ConservedVariables`, which is an aggregate data structure consisting of multiple *conserved variables* (see Section 4) that in turn consist of a set of moments of some underlying distribution function (such as density, momentum, etc.). During a simulation each block works only on its interior zones, but the reconstruction process requires that the data for the ghost zones must also be available. The ghost zones therefore must be filled by the blocks that have these zones in their interiors in a procedure known as *boundary exchange*.

A neighbor of some block (conventionally called “my block”) can be one of 5 different kinds, defined as the enumeration type `NeighborType`. A t-face neighbor (`GEONBR_TFACE`) is a block below or above my block that has the same sector index. An r-face neighbor (`GEONBR_RFACE`) is a block in the same slab, but a different sector that shares the r-face with my block. A t-edge neighbor (`GEONBR_TEDGE`) is a block above or below that has a common t-edge. Likewise, an r-edge neighbor (`GEONBR_REDGE`) is in the same slab, but only shares an r-edge with my block. Finally, the vertex neighbor (`GEONBR_VERTEX`) is in a different slab and only shares a single vertex with my block. Figure 14 shows a triangular block surrounding by ghost zones that belong to the interior of one of the neighbors. Note that a block serves the role of each neighbor type at some stage in the boundary exchange.

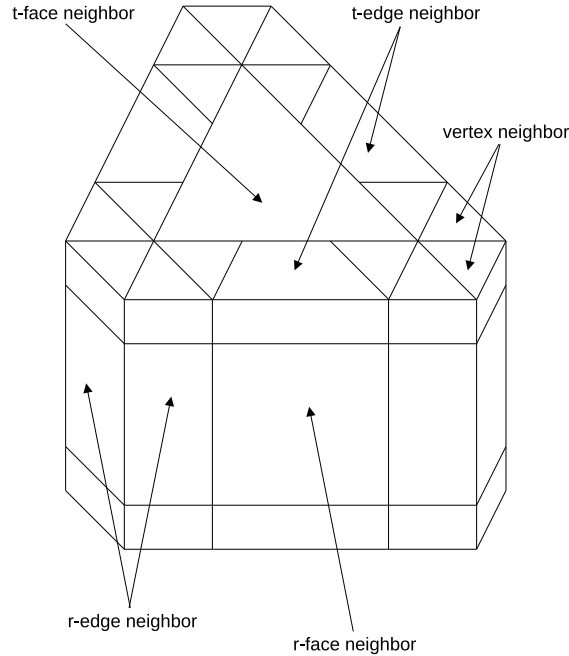


Figure 14: Face, edge, and vertex neighbors of a triangular block.

The boundary exchange process is based on the concept of an *exchange site* that serves as the above-mentioned intermediary. An abstract implementation of an exchange site (with no relationship to the actual

mesh) is defined in the header file `common/exchange_site.hh`. This file introduces a structure named `ExchangeSite` that is used to perform all messaging operations using MPI. The class provides buffer storage where grid blocks store the data from their interior and retrieve the data for the ghost zones after an exchange. Each block has a participation order that identifies it as the member of the exchange.

Although there is no specific “type” property of an exchange site, the sites used by the block are grouped into five categories corresponding to each neighbor type. The global function `ExchangeSiteCount()` calculates an array of exchange site counts, one element per neighbor type. These numbers are also stored locally in `exch_site_count`. It is easily seen that the numbers are 2, p , $2p$, p , and $2p$ for the five types, respectively. The controlling function (typically, a member function of an object of type `DomainPartition`), creates one `ExchangeSite` object per site and assigns a *participation order* to each block. It will create a subset of these objects for each block on the process and communicate them to the block by calling the public method `ImportExchangeSites()`, once per neighbor type. Exchange sites are created by the simulation level code that has access to the global topology; more details can be found the following section. Figure 15 presents a full set of 20 exchange sites for triangular blocks. By contrast, quadrilateral blocks have 26 such sites.

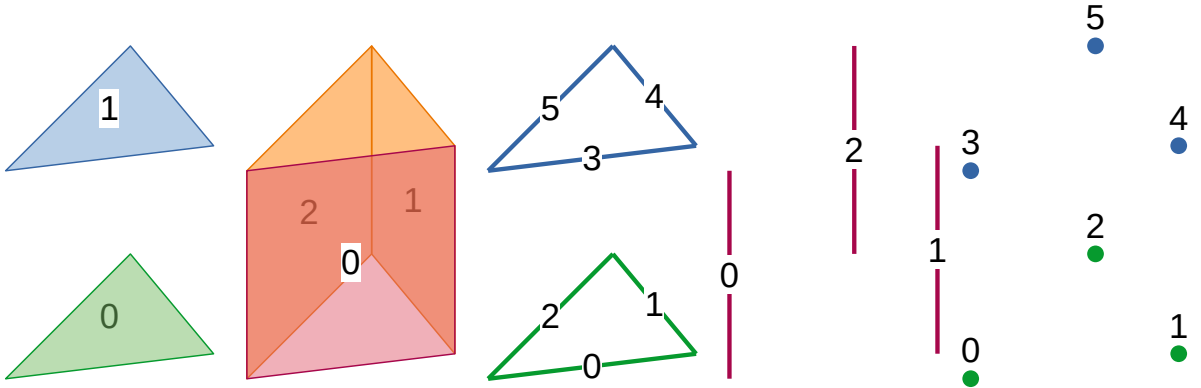


Figure 15: Numbering of exchange sites in a TAS block. From left to right: t-face, r-face, t-edge, r-edge, and vertex exchange sites.

The *default* number of participants for each site type can be obtained by calling `ExchangePartCount()`. The *actual* number of participants can be smaller if an exchange site contains the boundary of the simulation or a singular r-edge or vertex. The actual number is retrieved by accessing the exchange site’s variable `n.parts`. The block is already aware of the defects in the topology from the content of the arrays `border_type` and `corner_type`, so the number of participants can also be deduced at the local level.

In principle, it would be possible to exchange the messages directly in and out of the conserved variable storage `cons_vars` by setting up the appropriate MPI datatypes. However, the storage pattern is highly irregular, and it is more efficient to use dedicated buffer space where information is stored in a contiguous manner. The code used *translation tables* to map the face indices from the buffer space to the block space that are stored in `buf_face_translation`. In addition to that map each site/part combination has a starting shell (`buf_shell_start`). All buffers of the same type are identical in size; the arrays `buf_height`, `buf_length`, `buf_width`, `buf_area`, and `buf_volume` store their memory extents. The code must *pack* the data into the buffer before performing a send. Likewise, the receiving process must likewise *unpack* the receive buffer into `cons_vars`.

Figure 16 shows the extents of the buffers on the sphere. The side buffer is in the shape of a trapezoid in the TAS and a rectangle in the QAS, while the corner buffer is in the shape of a triangle or a square. The dot marks the origin of the indices (i_0, j_0) in each sub-block. It is calculated relative to the nearest corner of the sector and is therefore the same for the 6 (or 4) blocks that meet at that corner.

Generating the face maps is a complicated operation, which begins in `SetDimensions()`, where the memory for the buffer maps is allocated. As shown in Figure 16, the buffer sub-blocks can have 6 different orientations in the TAS and 4 in the QAS. Also, the origin of the coordinates changes depending on the

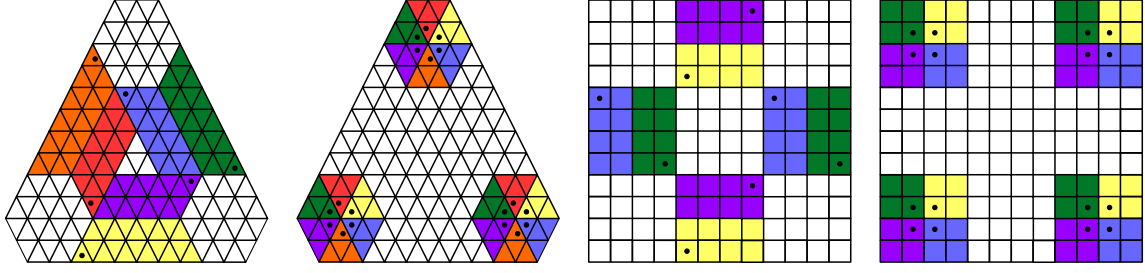


Figure 16: Ghost zone sub-block (i.e., buffer) orientation for the block neighbors in the TAS (two left panels) and QAS (two right panels) for side and corner neighbors. Different colors correspond to different orientations of the sub-block, and the dots show the origin of the coordinate system for the i_r and j_r indices. Note that the TAS send buffers in the first panel overlap at the corners of the block interior.

exchange site. The calculation consists of applying a rotation and a translation to the *base vertex* of the sub-block as described above. The results are stored in `buf_face_translation`.

The data member `buf_face_translation` is a 4-dimensional array whose indexing determines all exchange operations. The longest dimension is the neighbor type, the next is the exchange site, the third is the participant at that site, and the final is the face index. There is some redundancy because, for example, the maps of the vertex neighbor buffers are identical to those used for r-edge neighbors. `SetDimensions()` avoids duplication by using pointers where appropriate. Similar considerations are used to allocate space for `buf_shell_start`. Initially the part indexing of these maps corresponds to a full contingent of participants, i.e., a defect-free mesh. Figure 17 shows an example of how pointers to the arrays storing the face maps are arranged for a full set of exchange sites of a QGM block. Note that for certain neighbor types the pointer tables have quadruple redundancy. The `GEONBR_RFACE` and `GEONBR_REDEGE` pointers correspond to the first (shaded) columns in the tables; in those two cases there is no redundancy.

The public member function `ImportExchangeSites()` of the buffered block class is used to provide the block with a vector of shared pointers to the subset of all exchange sites that have this block as a participant in the second argument (the first argument, `ntype`, specifies the type of the exchange). After receiving the list, the function computes the buffer face translation tables if `ntype` is any of `GEONBR_TFACE`, `GEONBR_RFACE`, and `GEONBR_REDEGE` (the other maps are only pointers).

The storage pattern must be modified at a site represented by a singular corner. From the perspective of the exchange site, there are only 5/3 participants, not the standard 6/4. This means the face maps must be re-arranged to relocate the hole to the end of the array. This is performed in `ImportExchangeSites()` when the latter is called with the first argument equal to `GEONBR_REDEGE`. The relocation pattern depends on the value of `my_part`, the participation number of the block. The missing part is always opposite to `my_part` and can be obtained as $(\text{my_part} + \text{edges_per_vert}) \% \text{edges_per_vert}$. Table 3 shows how the actual maps (synchronized with the exchange site's numbering scheme) are arranged relative to the standard maps.

Table 3: Mapping between the actual parts (top row) and the standard parts (bottom row) for all possible values of `my_part` at a singular corner of the TGM.

	<code>my_part == 0</code>	<code>my_part == 1</code>	<code>my_part == 2</code>	<code>my_part == 3</code>	<code>my_part == 4</code>
actual	0 1 2 3 4 5	0 1 2 3 4 5	0 1 2 3 4 5	0 1 2 3 4 5	0 1 2 3 4 5
standard	0 1 2 4 5 3	0 1 2 3 5 4	0 1 2 3 4 5	5 1 2 3 4 0	5 0 2 3 4 1

0 / 0	0 / 2	4 / 0	4 / 2
0 / 1	0 / 3	4 / 1	4 / 3
1 / 0	1 / 2	5 / 0	5 / 2
1 / 1	1 / 3	5 / 1	5 / 3
2 / 0	2 / 2	6 / 0	6 / 2
2 / 1	2 / 3	6 / 1	6 / 3
3 / 0	3 / 2	7 / 0	7 / 2
3 / 1	3 / 3	7 / 1	7 / 3

edge 0, face 0
edge 0, face 1
edge 1, face 0
edge 1, face 1
edge 2, face 0
edge 2, face 1
edge 3, face 0
edge 3, face 1

0 / 0	0 / 1	1 / 0	1 / 1
-------	-------	-------	-------

face

0 / 0	0 / 4	4 / 0	4 / 4
0 / 1	0 / 5	4 / 1	4 / 5
0 / 2	0 / 6	4 / 2	4 / 6
0 / 3	0 / 7	4 / 3	4 / 7
1 / 0	1 / 4	5 / 0	5 / 4
1 / 1	1 / 5	5 / 1	5 / 5
1 / 2	1 / 6	5 / 2	5 / 6
1 / 3	1 / 7	5 / 3	5 / 7
2 / 0	2 / 4	6 / 0	6 / 4
2 / 1	2 / 5	6 / 1	6 / 5
2 / 2	2 / 6	6 / 2	6 / 6
2 / 3	2 / 7	6 / 3	6 / 7
3 / 0	3 / 4	7 / 0	7 / 4
3 / 1	3 / 5	7 / 1	7 / 5
3 / 2	3 / 6	7 / 2	7 / 6
3 / 3	3 / 7	7 / 3	7 / 7

vert 0, face 0
vert 0, face 1
vert 0, face 2
vert 0, face 3
vert 1, face 0
vert 1, face 1
vert 1, face 2
vert 1, face 3
vert 2, face 0
vert 2, face 1
vert 2, face 2
vert 2, face 3
vert 3, face 0
vert 3, face 1
vert 3, face 2
vert 3, face 3

Figure 17: Memory storage pattern for the face maps of all five neighbor types on a QGM mesh. Each diagram consists of two tables. The single column table on the right shows the unique maps (1, 8, and 16), while the table on the left shows the pointer arrangement. The `GEONBR_RFACE` and `GEONBR_REDEGE` have one to one mapping (grey shaded areas), while `GEONBR_TFACE`, `GEONBR_TEDGE` and `GEONBR_VERTEX` have quadruple redundancy. The top left mapping is for `GEONBR_RFACE` and `GEONBR_TEDGE`, the bottom left mapping is for `GEONBR_TFACE`, and the rightmost mapping is for `GEONBR_REDEGE` and `GEONBR_VERTEX` neighbor types. The numbers in the left tables are the site/part pairs.

3.8 Grid block class hierarchy

The relationship between the classes in the grid block family is shown in Figure 18. The final derived class, `SimulationBlock`, is described in the next section because it is more closely related to the physics of the problem than to the geometry of the mesh, as was the case up to this point. A top level class representing the entire simulation should include a vector of simulation blocks that are processed sequentially on each process and in parallel between the processes. Each block is an autonomous object that is completely oblivious to the existence of other blocks, and can perform almost all of its tasks unsupervised.

4 Performing a simulation

A simulation is a procedure of evolving a state of the entire system under investigation in time. The state is characterized by a set of conservation laws (CLs) that are themselves collections of conserved variables (density, momentum, energy, etc.). The conservation laws are coupled with other either by sharing a variable (such as velocity or magnetic field) or by exchanging information via source terms. Throughout a simulation the CLs are stored in a contiguous data structure of the type **specify which type**. A conservation law can be active or passive. An active CL evolves on its own accord, typically under the effects of volumetric forces acting on a fluid element. Passive CL evolution, on the other hand, is controlled by an active CL it is linked with. A typical example is a flow of partially ionized plasma with an admixture of energetic ions. The system can be described using MHD (magnetohydrodynamic), PUI (pickup ion), GASDYN (gas-dynamic) and IV (indicator variable) CLs. The active MHD CL contains the bulk plasma density, momentum, energy, and magnetic field, while the passive PUI CL contains the ion density and energy, momentum being determined from the bulk velocity from the MHD CL. An IV CL (passive) consists of a single variable slaved to the MHD CL. GASDYN variables are active and contain a complete set of fluid variables (without the magnetic

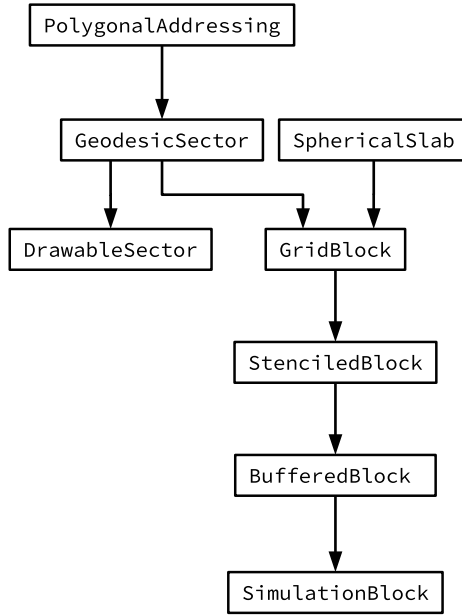


Figure 18: The grid block class hierarchy in SPECTRUM.

field). Figure 19 shows the structure of a state variable with the arrows showing the connections between the CLs. A one way arrow signifies an active-passive connection, while a double-sided arrow marks a two-way coupling.

We distinguish three kinds of states for CLs: primitive, conserved, and flux. Primitive variables are density, velocity, pressure (or temperature), and magnetic field. They characterize the internal state of the fluid and cannot be added together (e.g., it makes no sense to add two temperature together). Conserved variables, on the other hand, are extrinsic, so they add up if the corresponding fluid elements are combined. Primitive variables are density, momentum, energy, and magnetic field. Fluxes give the rate of flow of each conserved quantity in or out of a volume. Their names are obtained by taking a conserved variable and adding “flux” to it, as in momentum flux. Any kind of variable can be readily converted to the other two kinds via conversion routines.

During the course of a simulation the entire state is stored in contiguous memory in the array `zone_cons` of each block. During processing a state is separated into individual CLs, and each is subjected to the same set of operations that comprise a finite volume method, such as reconstruction, flux calculation, or time update. The next section describes the process in more detail.

4.1 Introduction to finite volume methods

The continuum plasma sub-system of the SPECTRUM code is intended as an extensible framework capable of dealing with different fluid-type models describing a multi-component plasma. The core concept is a finite-volume method, where all physical quantities are defined as averages over a zone. The change of a quantity in a zone over time is then due to either a “flow” of the associated variable through the surfaces of the zone, or a local source or sink. A CL system of size n ($=$ the number of scalars in the system, where each vector counts as three) can be written in a general form as

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot \mathbf{F} = \mathbf{S}, \quad (16)$$

where $\mathbf{U}$ is the n -vector of conserved variables, $\mathbf{F}$ is the flux n -vector, and $\mathbf{S}$ is the source n -vector. The divergence in (16) is the ordinary (Cartesian) space divergence. For the MHD system the conserved 8-vector

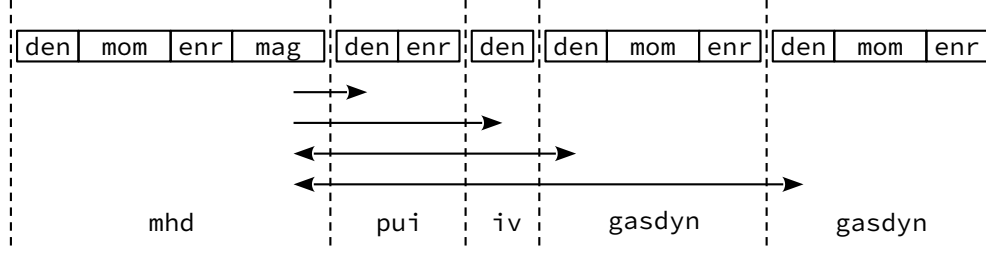


Figure 19: A complete simulation state consisting of five conservation laws: an MHD CL describing the bulk plasma, a PUI CL describing an energetic particle component, an IV CL for a passively advected variable, and two GASDYN CLs describing neutral hydrogen components. The PUI and IV CLs are passive and linked to the MHD CL. The two GASDYN CLs are bi-directionally coupled to the MHD CL via source terms. Vector variables are represented by a bar with a larger width.

is

$$\mathbf{U} = (\rho, \rho u_x, \rho u_y, \rho u_z, e, B_x, B_y, B_z)^T, \quad (17)$$

where ρ is density, $\mathbf{u}$ is velocity, e is energy density, and $\mathbf{B}$ is magnetic field. The expression “n-vector” is used to distinguish the CL variables from ordinary 3-vectors. Note that the flux $\mathbf{F}$ is a tensor object.

The system (16) can be written in a quasi-linear matrix form by using the transformation $d\mathbf{F} = (\partial\mathbf{F}/\partial\mathbf{U})d\mathbf{U}$. The properties of the Jacobi matrix $(\partial\mathbf{F}/\partial\mathbf{U})$ of this transformation determine the system type. In a hyperbolic system, in particular, all eigenvalues of the Jacobi matrix are real. The methods described in this section apply to hyperbolic systems only. Equations of gas dynamics and magnetohydrodynamics (MHD) are hyperbolic, but some other systems, such as CGL-MHD, have regions in their variable space where they lose hyperbolicity and become elliptic, so the numerical methods used for hyperbolic systems are bound to fail. For such systems additional techniques must be used to prevent the system from venturing into the elliptic regions.

Consider a zone addressed by a shell/face pair of indices (l, f) . Suppose the unit-sphere area of the t-face f is Ω_f , and the unit-circle length of the t-edge e is φ_e . For a geodesic spherical mesh we can write a zone averaged quantity as

$$\langle \mathbf{U} \rangle_{lf} = \frac{1}{r_l \bar{r}_l \Delta_l \Omega_f} \int_{\Omega_f} \int_{r_{l-1/2}}^{r_{l+1/2}} \mathbf{U}(\mathbf{r}) r^2 dr d\Omega, \quad (18)$$

where integration in the solid angle Ω is equivalent to an integration over the surface of a unit sphere, and $\bar{r}_l$ is given by eq. (66) of Appendix C. Averaging (16) over the volume of the zone and using Gauss’s theorem on the divergence yields the semi-discrete version of the conservation law:

$$\frac{\partial \bar{\mathbf{U}}_{lf}}{\partial t} = -\frac{1}{r_l \bar{r}_l \Delta_l} \left(r_{l+1/2}^2 \bar{\mathbf{F}}_{r,l+1/2,f} - r_{l-1/2}^2 \bar{\mathbf{F}}_{r,l-1/2,f} \right) - \frac{1}{\bar{r}_l \Omega_f} \sum_e \varphi_e \bar{\mathbf{F}}_{\tau,le} + \bar{\mathbf{S}}_{lf}. \quad (19)$$

The second term on the RHS describes the fluxes through the top and bottom t-faces, while the third term is the sum of the fluxes through the r-faces. The subscript r , used for the fluxes, indicates a projection onto the radial normal, while the subscript τ designates a projection onto a vector tangent to the sphere and normal to the corresponding t-edge. Care must be taken to ensure that the τ normal direction is in the outward sense.

Time integration in (19) is performed with a Runge-Kutta method. For second order accuracy in time the midpoint method (`RK_INTEGRATOR.TYPE = 3`) can be used. The fluxes are evaluated as solutions to the Riemann problem (RP) at each interface. A RP consists of two constant states (fully specified vectors $\mathbf{U}$) known typically as left and right states that are separated by a solid membrane. At one instant in time the membrane is removed and a pattern of nonlinear waves called *simple waves* develops that evolves in a self-similar fashion, where each quantity depends only on the ratio x/t . Exact RP solutions are known only

for certain hyperbolic systems. However, such solutions often possess some undesirable properties, such as a lack of convexity. For a number of reasons approximate numerical solutions are commonly used that result in a better stability of the overall numerical scheme.

The RP requires that either conserved or primitive variables are known on each side of the interface, which leads us back to the concept of reconstruction. As mentioned earlier, in this process a sub-grid functional representation of each variable is built based on the knowledge of the variable in adjacent zones. Subsequently, the face averaged fluxes must be evaluated that incurs integration over the faces. Integration is performed using numerical quadratures of the order (typically) sufficient to exactly integrate a reconstruction polynomial. Note that all integrals are separable into integration on the unit sphere (where each quantity will bear the subscript “u”) times one-dimensional integration in the radial direction.

SPECTRUM uses only second-order in space continuum methods, which means the reconstruction polynomial is a linear function. Such a function requires only a single quadrature point located in the center of mass of the face. For flat r-faces the center is given by the center of mass of a unit-circle arc returned by the function `GeoVector::CircArcCenter()` scaled by a factor of $\bar{r}_l$. For curved t-faces we use the center of the spherical triangle that can be computed by calling `GeoVector::SphTriCenter()`, without scaling. One should be aware that the mass center does not lie on the sphere. This means that for t-faces the reconstruction should be performed to a point *internal* to the lower zone and *external* to the upper zone, not the interface!

4.2 Performing the reconstruction

The terminal class of the grid block family is called `SimulationBlock`. Derived from `BufferedBlock`, this class adds the capability to perform variable reconstruction, solve the RP, compute the fluxes, and update the conserved variables at each time step. The class is declared in `simulation_block.hh` and its implementation may be found in `simulation_block.cc`. At the beginning of a simulation all conserved variables must be initialized with a user provided functional form (for time dependent boundary conditions such an operation must be also performed at each time step for blocks that include the boundary of the simulation domain). An object of the polymorphic class `BackgroundBase` is used for initialization. As for the faces, volume averaging is done using numerical quadratures. Again, if only linear functions are considered, a single quadrature point at the geometric center of the zone will suffice. That point is the mass center of the spherical triangle scaled by a factor of $\bar{r}_l$ given by eq. (69) of Appendix B.

The sub-grid reconstruction of the variable $\mathbf{U}$ using linear functions is given by eq. (15). Reconstruction of the conserved variables is performed by calling the public method of `SimulationBlock` named `Reconstruct()` which makes use of many of the stenciled block’s facilities. This step is independent of the physics of the problem, so we may concentrate on one particular component of the vector $\mathbf{U}$, which will be written simply as U . Reconstruction is performed on *multiple* stencils (s) and the modes from each stencil are weighted-averaged to yield a *limited* reconstruction. After the averaging, the reconstruction polynomial is evaluated in the centers of every face to obtain the face-average values. The first-order modes themselves are not saved. The face variables are stored in a three-dimensional array `face_cons`. As with conserved variables, the first index is the shell and the second is the face. The third index enumerates the faces of the frustum: 0 and 1 refer to the bottom and top t-faces, and the remaining p entries are the r-faces.

In reality, because of the shell-face factoring, reconstruction is performed in *exponential coordinates*, defined in the first sub-section of Appendix A. Define a new variable,

$$\mathbf{W}(\mathbf{r}') = \frac{r^2}{r'^3} \frac{dr}{d\xi} \mathbf{U}(\mathbf{r}), \quad (20)$$

where r is the actual coordinate and r' is the exponential coordinate (the result of an application of the exponential mapping function to the same index variable ξ). Instead of reconstructing U we will reconstruct W and then perform the conversion. Reconstruction in the zone (l, f) using stencil s can be written as

$$W_{lf}^s(\mathbf{r}) = \bar{W}_{lf} + \delta \mathbf{W}_{lf}^s \cdot (\boldsymbol{\rho} - \bar{\boldsymbol{\rho}}_f), \quad (21)$$

where $\boldsymbol{\rho} = \mathbf{r}'/r'_s$. Observe that equation (73) has no terms explicitly dependent on the the shell index l , so the reconstruction in all zones sharing the face index f can be performed using the same matrix, which leads to a substantial storage saving.

The reconstruction procedure is performed for every stencil. There are 7 and 9 stencils for the TGM and QGM, respectively. A weighted average process is performed to *limit* the reconstruction to prevent the reconstructed variables to assume very large or very small values at the edges of the principal zone. Having such large variations leads to numerical instability development near discontinuities that must be avoided. The standard approach is called WENO (weighted essentially non-oscillatory) method. In this method a weighted average of the coefficients $\delta \mathbf{W}_{lf}^s$ is computed,

$$\delta \mathbf{W}_{lf} = \sum_s \bar{w}^s \delta \mathbf{W}_{lf}^s. \quad (22)$$

The normalized weights $\bar{w}^s$ depend on how much variation each stencil admits. A large variation results in a smaller weight. The stencils are also biased, so that the central stencil receives a larger weight than the directional stencils. Suppose the biases add up to a unity. The value of γ^0 is stored in the global constant `bias_central` in the file `simulation_block.hh`. Typically $\gamma^0 > 0.8$, so the rest of the stencils have $\gamma^i < 0.033$. The weights are computed as

$$w^s = \frac{\gamma^s}{[(\Delta_l \delta \mathbf{W}_{lf}^s)^2 + (\epsilon W_{lf})^2]^2}. \quad (23)$$

Here ϵ is a safety factor (small number) to avoid a division by zero. It is stored in the constant `weno_safety`.

4.3 Conservation laws

The fluxes in eq. (19) are evaluated from the solution of the RP with the left and right reconstructed and limited states as inputs. This step is highly physics-dependent. The physics-dependent code is located in the `fluid` sub-directory of the SPECTRUM source tree. This directory contains several header files whose name contains the prefix “conservation_laws”. Each one implements a different physics represented by a set of three classes for the conserved variables, the primitive variables, and the flux function. Each class inherits from `SimpleArray` and includes no other storage. In addition to the default constructors the first two classes implement two conversion routines, `ToPrimitive()` and `ToFlux()` for the conserved and `ToConserved()` and `ToFlux()` for the primitive. Each class includes a static boolean variable `is_cons_law_active` that marks it as eligible to be used with a Riemann solver.

The simplest conservation law is that of a passively advected scalar, which is implemented in the header `conservation_laws_passive.hh`. The corresponding family of classes representing primitive and conserved variables and fluxes are grouped together as nested classes of the `Primitive` class. Their names are the same for every conservation law enclosing class and follow the same pattern, `PrimitiveState`, `ConservedState`, and `FluxFunction`. The class `Passive` has its type trait `is_cons_law_active` set to `false`, so very little needs to be implemented. The conversion routines are templated with a template argument being a class of “active” conservation laws. For the latter the conversion routines are not templates, but full implementations in each class.

Two additional files (`conservation_laws_gasdyn.hh` and `conservation_laws_mhd.hh`) contain the physics of compressible gas dynamics and MHD, respectively. The enclosing classes are actually templates taking one argument, `fluid`, which selects the fluid index (different fluids could have, for example, different specific heat ratios). The enclosing classes `Gasdyn` and `MHD` are marked as “active”, as such their nested classes define additional methods, such as `FastestWave()`, that calculates the largest wave speed in the system (e.g., fast magnetosonic wave for MHD), `Invert()` to reflect the vector quantities such as velocity or magnetic field, and several more specific functions used by Riemann solver classes. Multiple variables are grouped together into a single class called `ConservedVariables`, defined in the header `variables.hh`. There is typically one primary variable, a set of secondary variables, and a set of passive variables. For example, in a partially ionized plasma the primary variable would be an ionized component (MHD), secondary variables would be neutral components (gas dynamic) and the indicator variables (passive) marking different regions in the simulations can also be present.

The file `riemann_solver.hh` implements several common Riemann solvers. The base class template `RiemannSolverBase` defines the input (left and right variables) and the output (resolved fluxes and states). These are given intuitive names such as `left_prim` and `right_flux`. A solver can function using either

primitive or conserved states as the input. The `Solve()` method is the public interface function to obtain a RP solution for a given set of inputs (either primitive or conserved). After storing the input, the function calls a protected method `SolveInternal()` which does the actual work. The result must be requested separately by calling `GetResolvedPrim()`, `GetResolvedCons()`, or `GetResolvedFlux()` depending on the user's needs. There are four solvers currently defined, the simple Rusanov solver (`RiemannSolverRusanov`), the HLLC solver (`RiemannSolverHLLC`), the HLLC solver (`RiemannSolverHLLC`), and the special solver (`RiemannSolverNorelect`) used in certain types of boundary conditions. The solvers themselves are very simple and can be applied to any type of an active conservation law, used as the template argument `ConsLaw`, so they rely on some of the functionality of the conservation law classes. The latter includes methods such as `FastestWaveNormal()`, `FixEnergy()`, `FixMomentum()`, and `ToSonic()`.

4.4 Computational cycle

A single time step contains an outer loop on RK stages. There is one complete set of conserved variables per stage. Each stage consists of the following sub-steps:

1. Apply the Dirichlet boundary conditions. Ghost zones of the blocks that contain a physical boundary are updated. If the BC's are time independent, this step is omitted. This step is implemented in `FillBoundary()`.
2. Fill the owned buffers of every exchange site. This is performed by calling `PackBuffers()` in a loop over the neighbor type.
3. Exchange the boundary zones between the blocks by calling each exchange site's `Exchange()` method. Blocks must not call this method because exchange is ordered by ranks rather than blocks! This is the responsibility of a simulation level object.
4. Retrieve data for our ghost zones from the buffers by calling `UnPackBuffers()`, also in a loop over the neighbor type.
5. Perform reconstruction in every zone (except the outermost layer) on every stencil and obtaining their weighted (WENO) averages. This produces a set of $p + 2$ face averaged conserved variables, stored in `face_cons`.
6. Determine the local time step. Each block does it by calling its `TimeStep()` method and stores the time step in the block's shared site object `timestep_site` of the type `SharedSite`.
7. Globally reduce the time step by calling the `Share()` method of the `timestep_site`. As with the boundary exchange, blocks should not call this method leaving it to the simulation level object to do the reduction.
8. Retrieve the time step from the shared site.
9. Solve the RP at every t-face and r-face in the mesh (except the outermost layer) and update the variables in the two zones sharing this face. The two zones have the same flux added but with a different sign, so that the total quantity is strictly conserved. This operation is performed in the method `TimeUpdate()`.

5 Running in parallel

SPECTRUM is designed to be a highly parallel framework. The fluid subsystem is based on the distributed memory model and is implemented through the MPI library. MPI communications between processes residing in the same shared memory node happen at the speed of memory access, so no additional performance would be gained if a hybrid shared-distributed memory model was used instead. For this reason only the master and worker roles have a meaning – there is no need for a server. Because file I/O is also parallel, the master's only role is to generate informational and error messages. Those take negligible time, so the master always performs the functions of a worker.

5.1 Abstracting communication from the blocks

As described above, the computational unit of the mesh in SPECTRUM is a grid block. A process can have any number of grid blocks in it (potentially, all of them in a serial run). Grid blocks interact with each other via objects known as *communication sites*. According to the OOP philosophy, a block should have only a bare minimum information about the other blocks. Here all such information is contained inside an object derived

from the base template `CommunicationSite` defined in the file `communication_site.hh` (all site related files are currently in the `common` directory). The class contains an MPI communicator `site.comm` that includes only the processes that have blocks participating in this site. Each block is assigned a *participation order number* that uniquely identifies it as a member of the communication. The class also allocates buffer space where each participant stores the local data it wants to share, and retrieves the data shared by the other participants. The communication site template has one parameter, `datatype`, that corresponds to the type of data being communicated.

Two additional classes derive from `CommunicationSite`. The first, `SharedSite`, is defined in the header file `shared_site.hh`. Its purpose is to perform a global reduction operation on data from all site members and distribute the resulting values. Its main application is to compute the global (minimum) time step for the entire simulation. The file defines three function template, `MPI_User_Min()`, `MPI_User_Max()`, and `MPI_User_Sum()` that perform a comparison between two arrays of type `mpi_datatype` (the fourth argument), `first` and `second` (first and second arguments) of length `length` (third argument). These functions are used to create a new MPI reduction operation using `MPI_Op_create()`. The operations currently implemented (minimum, maximum, and sum) correspond to pre-defined MPI ops, but they can be applied to an arbitrary type that has `<`, `>`, and `+` operators defined. Developers can easily add additional operations if needed.

`SharedSite` is a class template derived from `CommunicationSite`. It adds a second template argument (`op`) which is an integer. Values of 0, 1, and 2 correspond to the minimum, maximum, and *average* operations, respectively. Copying of communication sites is not allowed because of MPI restrictions, so the copy constructor is deleted (the move constructor, however, does exist). A simulation level object typically contains a single shared site with template parameters `double` and 0. A shared pointer to the site is given to all the blocks. When the blocks have all finished placing their data into the buffer, the controlling function calls the method `Share()` that does three things. First, a reduction is performed among all the participants on the same process, second, a global reduction among all the processes is performed by calling `MPI_Allreduce()`, and third, the result is processed (division by the number of participants is required for the average) and placed in the buffer for retrieval by the participants.

The second derived class template is named `ExchangeSite` (defined in `exchange_site.hh`). Unlike `SharedSite`, this class does not reduce the data, but instead distributes it among the participants in such a way that all of them have access to the entire data. This is accomplished by a single call to `MPI_Allgatherv()`. Exchange sites are used to exchange the boundary information between blocks to fill their ghost zones. The following rules describe how the elements are numbered within a site.

1. The default number of blocks participating in an exchange is equal to 2, 2, 4, q , and $2q$ for the five neighbor types, respectively. A site of type `GEONBR_TFACE`, `GEONBR_TEDGE`, or `GEONBR_VERTEX` that lies on the simulation boundary has *one half* of the default number of participants (1, 2, and q , respectively). A site of type `GEONBR_REEDGE` and `GEONBR_VERTEX` that lies on a singular corner has one and two fewer participants ($q - 1$ and $2q - 2$) respectively. A `GEONBR_VERTEX` site subject to both conditions has $q - 1$ participants.

2. Participants with a lower slab index have lower participation order than the participants with a higher slab index.

3. Participants with the same slab index and different sector index enter in the order of their sector in the appropriate tessellation connectivity list. For `GEONBR_RFACE` and `GEONBR_TEDGE` sites it is the edge-face neighbor array, and for `GEONBR_REEDGE` and `GEONBR_VERTEX` it is the vertex-face neighbor array.

Figure 20 demonstrates the numbering scheme for a `GEONBR_REEDGE` (left) and a `GEONBR_TEDGE` (right) sites. For the former, the order of the sector in the vertex-face connectivity list determines the participation number. In the latter, participants 0 and 1 are lower-slab blocks in their order of occurrence in the edge-face list, and participants 2 and 3 are blocks directly above 0 and 1. If the `GEONBR_TEDGE` site is at the simulation boundary, only parts 0 and 1 are present.

An object derived from `CommunicationSite` is not aware of the mesh; it only records the part numbers and their ordering supplied by the calling function. Two `std::map` objects are provided that can be used by the blocks to perform an exchange. The first is the `part_lookup` map that returns the participant number given a unique “label”. SPECTRUM uses block indices as labels, but this is not a requirement as far as the class is concerned. Since a block always knows its index, the corresponding part can be readily found as `site.part_lookup[block_index]`, where `site` is the instance of `CommunicationSite` corresponding to the site in question. The second map, called `buffer_entry`, returns the address of the buffer space where the block can store and retrieve information before and after the communication by calling `PackBuffers()`

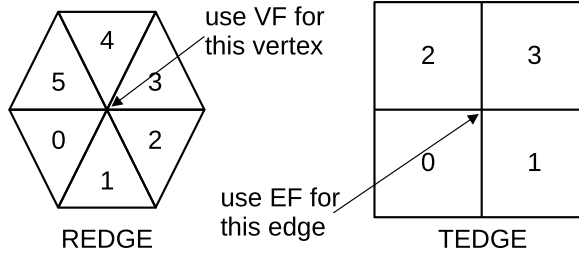


Figure 20: Ordering of blocks in an exchange site of type `GEONBR_REDGE` (left) and a `GEONBR_TEDGE` (right). The number shown inside a block corresponds to its index (part number) in the exchange.

and `UnPackBuffers()`, respectively. This map is required because the order of the blocks in the buffer space *differs* from the part order. This is necessary because each process must communicate a single contiguous chunk of the global buffer. For example, if a process has parts 3 and 8 (disjoint), their buffer spaces will be located next to each other in the global buffer (perhaps as entries 3 and 4). If `site_comm` has a single rank in it, then the entirety of data is already available in the buffer, where each participant can retrieve it directly, and an MPI collective call is omitted.

The intended use of `CommunicationSite` is to create an array of objects, one per site, on each process. This appears hugely wasteful because a process would in many cases have access to only a subset of all sites. For example, consider an icosahedral mesh topology consisting of level 0 sectors (of which there are 20) and 10 shells for the total of 200 blocks. According to Table 1 there are $20 \times 11 = 220$ t-face sites, $30 \times 10 = 300$ r-face sites, $30 \times 11 = 330$ t-edge sites, $12 \times 10 = 120$ r-edge sites, and $12 \times 11 = 132$ vertex sites, for the total of 1102 sites. Each TGM block participates in 20 exchange sites (2, 3, 6, 3, and 6 of the five neighbor types, respectively), which is less than 2% of the total. The rationale for the design is that the memory footprint of the class in its un-initialized state (without buffers) is small (under 200 bytes), and the benefits it offers by abstracting all MPI communications away from grid blocks outweighs the increase in memory use. Remember that the MPI communicator creation and destruction routines must run on all processes, including those that are *not* in the site communicator. The initialization is performed by calling the public member `SetUpProperties()`, which accepts the site index, the number of participants, the size of one buffer chunk (per participant), the list of ranks, and the list of labels as its arguments. This routine will generate a list of unique ranks and create a communicator for them.

On non-participating ranks, no further action is taken and the function exits. On the ranks with participating blocks, arrays of send/receive counts and displacements are generated, and then the two maps described above are calculated. The file `common/exchange_site.hh` includes a test function `TestExchange()` that can be used to verify most of the class's functionality. This function simulates an exchange by generating a simple topology with 8 participants and random distribution of participants among the processes. Typically some processes will have no participants, some will have a single participant, and some will have two or more. The code will print the site's buffer before and after the exchange. Before the exchange the buffer contains only locally available data, and after the exchange it should be filled in the correct order. The main program to perform this testing is found in `main_test_exchange_site.cc`. `TestExchange()` takes one parameter, which is the rank that will print the diagnostic.

5.2 Domain topology

The class template `DomainPartition`, declared in the header `geodesic/domain_partition.hh` and implemented in `domain_partition.cc`, is a simulation level object that controls domain decomposition and assignment of tasks to processes. Domain decomposition is automatic: the user only specifies the face division d and the number of shells H (and of course the number of processes as an argument to `mpirun`), and the code calculates the rest. The code should pick an optimal block configuration (the sector division d_s and the number of slabs N) based on the following considerations: (a) the block length should be similar to the block height, (b) larger blocks are preferred to smaller blocks, and (c) the number of blocks on each

process should be the same (if possible). The number of shells might not be divisible by N , in which case some of the slabs will have one less shell than others. For example, if $H = 347$ and $N = 8$, we can write $347 = 5 \times 43 + 3 \times 44$. It is convenient to assign the thinner slabs to the blocks with smaller indices because the number of such slabs can never be zero (the number of slabs with larger indices can be zero if H is divisible by N). The numbers can be computed using integer math as follows:

```
int H = 347;
int N = 8;
int shells_per_slab = H / N; // smaller of the two
int n_slabs = N * (shells_per_slab + 1) - H; // number of smaller slabs
```

The sector dimensions are obtained as follows

```
#define POLY_TYPE POLY_TETRAHEDRON
#define MAX_FACE_DIVISION 7
int sector_div = 1;
int face_div = 6;
TraversableTessellation<POLY_TYPE, MAX_FACE_DIVISION> tessellation;
int n_sectors = tessellation.NFaces(sector_div);
int sector_width = pow2(face_div - sector_div);
```

The currently implemented partitioning strategy is to start with division 0 sectors and a single slab (this is called the minimum decomposition). If the number of blocks is smaller than the number of worker processes, then this will be the final topology. If this is not the case, then the code compares the length and width of the block and refines the larger dimension until the number of blocks exceeds the number of processes.

A process can have only a single instance of `DomainPartition`. The class has a vector `blocks_local` consisting of objects of the template type `blocktype`, where all computations are performed. The domain class owns a `TraversableTessellation` object that is used to calculate the block coordinates and the relationship between the blocks. It contains a polymorphic object of base type `DistanceBase` that stores the radial rationing of the shells. It also has a large array of objects of the type `ExchangeSite` (most of which are likely to be un-initialized blanks) named `exchange_sites`. The domain partition object assigns the participation orders to blocks in each exchange site where that blok is a participant, based on the rules described above using the information from the member tessellation object. The design assures that there is no conflict between messages and no deadlock occurs regardless of the number of blocks in the process. The downside is that the MPI implementation used must have a sufficiently high limit on the number of communicators (the historical value of 8196 used in early MPI implementations is clearly too low).

Block assignment to processes happens in the constructor using the round robin method. Two more public function calls are needed to complete the setup. The first is `SetUpExchangeSites()`, which initializes the array of `ExchangeSite` objects named `exch_sites`. This is done by looping over all the slabs (or slab interfaces) and the geodesic mesh elements that represent the exchange site's projection onto a sphere (faces, edges or vertices) and finding every neighbor of that element by calling the `RequestableTessellation` methods `FaceNeighborsOfEdge()` and `FaceNeighborsOfVert()`. The function also generates the vector `exch_sites_local` containing the indices of exchange sites where at least one of the local block is a participant. The second function is called `ExportExchangeSites()`, which generates a vector of shared pointers to exchange sites for each of owned blocks and communicates them to the blocks by calling the latter's `ImportExchangeSites()` function. An exchange typically occurs at a specified stage during every time step. It is accomplished with a call to `ExchangeAll()` which simply calls the `Exchange()` method of every local exchange site.

Appendix A: Vector operations in SPECTRUM

Many of the vector operations performed in the course of a simulation involve vectors of the same length (assumed to be of unit length for simplicity). In many situations these operations can be written in a vector form thus reducing the use of computationally expensive trigonometric functions. The functions described in this section are implemented in `vectors.hh` and `vectors.cc` that reside in the `common` directory. The

GeoVector class is basically a three-element array with a large number of operators and functions (both members and non-members) to make the coding faster and less error prone. Operators acting on vectors include multiplication and division by a scalar, addition and subtraction, component-wise multiplication and division, scalar product (**operator ***()), and vector product (**operator ^**()). Most operators can be applied from both the left and the right. There is also an important function **Normalize()** that scales a given vector to a unit vector. In mathematical expressions we will designate it with $[\mathbf{v}]_u = \mathbf{v}/v$. If one needs to generate a unit vector from another vector without changing the latter, a non-member function **UnitVec()** should be used.

Vector transformations include rotation and a change of basis. The former is performed by **GeoVector** member function **Rotate()** which takes two arguments, **axis** (the rotation axis $\hat{\mathbf{n}}$) and **angle**, the angle of rotation α measured in the counter-clockwise direction. The formula for rotation is

$$\mathbf{v}' = \cos \alpha \mathbf{v} + (1 - \cos \alpha)(\hat{\mathbf{n}} \cdot \mathbf{v})\hat{\mathbf{n}} + \sin \alpha(\hat{\mathbf{n}} \times \mathbf{v}). \quad (24)$$

A projection onto a plane is performed by **SubtractParallel()** that takes a single argument **axis**, the normal to the plane. Projection is mathematically expressed as

$$\mathbf{v}' = \mathbf{v} - (\hat{\mathbf{n}} \cdot \mathbf{v})\hat{\mathbf{n}}, \quad (25)$$

which explains the name of the function. A basis transformation is performed by a pair of complimentary functions **ChangeToBasis()** and **ChangeFromBasis()**. They both take one argument, an array of three vectors called **basis**, which is the basis $\hat{\mathbf{e}}'_i$ of the new frame. The forward coordinate transformation is given by

$$v'_i = \mathbf{v} \cdot \hat{\mathbf{e}}'_i, \quad (26)$$

while the inverse transformation is

$$v_i = \sum_j v'_j \hat{e}_{ji}, \quad (27)$$

There are also functions to convert between spherical and Cartesian coordinates named **ToSpherical()** and **ToCartesian()**. For efficiency of calculations they accept the sines and cosines of the polar and azimuthal angles of the position vector (**sintheta**, etc.) instead of the vector itself. This allows the calling program to perform multiple conversion for the same position vector without evaluating the trigonometric functions every time.

A pair of unit vectors defines a plane and a *central angle*, whose value is equal to the length of the great circle arc connecting the tips of the vectors. The normal to the plane is simply

$$\hat{\mathbf{n}} = [\mathbf{v}_1 \times \mathbf{v}_2]_u, \quad (28)$$

which can be computed by calling **Normalize()** on a vector product. The central angle (or arc length) is computed by **CircArcLength()** that accept two vector arguments and returns the angle as

$$\varphi = \cos^{-1}(\mathbf{v}_1 \cdot \mathbf{v}_2). \quad (29)$$

The center of mass of a circular arc can be computed with **CircArcCenter()** which accepts the same two arguments. The expression is

$$\mathbf{c} = \frac{2}{\varphi} \sin\left(\frac{\varphi}{2}\right) [\mathbf{v}_1 + \mathbf{v}_2]_u, \quad (30)$$

where φ is the arc length defined above. The center of mass does not lie on the unit sphere.

Two additional helper routines bisect an angle between two vectors (**Bisect()**) and divide an angle into two unequal parts. The latter is performed by the function **DivideAngle()** that takes three arguments, the two vectors and the number **frac** that specifies which fraction f of the total should the first angle (between the first vector and the new vector) be. The second angle (between the new vector and the second vector) is therefore $1 - f$ of the total. The new vector is calculated as

$$\mathbf{v} = \frac{\sin[(1 - f)\varphi]\mathbf{v}_1 + \sin(f\varphi)\mathbf{v}_2}{\sin \varphi}, \quad (31)$$

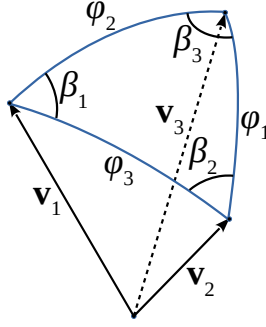


Figure 21: A spherical triangle defined by three unit vectors $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_3$ showing the central angles φ_i and the vertex angles β_i .

where φ is the central angle between $\mathbf{v}_1$ and $\mathbf{v}_2$ (29).

A set of three unit vectors defines a spherical triangle (Figure 21). The angles β_i of this triangle are called *vertex angles*. A vertex angle is defined by three vectors; it should not be confused with central angle that requires only two. Given $\mathbf{v}_1$, $\mathbf{v}_2$, and $\mathbf{v}_3$, the cosine of the vertex angle at $\mathbf{v}_2$ can be computed as

$$\cos \beta_2 = \frac{(\mathbf{v}_1 \cdot \mathbf{v}_3) - (\mathbf{v}_1 \cdot \mathbf{v}_2)(\mathbf{v}_2 \cdot \mathbf{v}_3)}{|\mathbf{v}_1 \times \mathbf{v}_2||\mathbf{v}_2 \times \mathbf{v}_3|}. \quad (32)$$

This equation is encoded in the function `VertexAngle()` with three `GeoVector` arguments returning a `double`. The area of a spherical triangle is

$$\Omega = 2 \cos^{-1} \frac{1 + \mathbf{v}_1 \cdot \mathbf{v}_2 + \mathbf{v}_2 \cdot \mathbf{v}_3 + \mathbf{v}_3 \cdot \mathbf{v}_1}{\sqrt{2(1 + \mathbf{v}_1 \cdot \mathbf{v}_2)(1 + \mathbf{v}_2 \cdot \mathbf{v}_3)(1 + \mathbf{v}_3 \cdot \mathbf{v}_1)}}, \quad (33)$$

implemented as `SphTriArea()`. The center of mass of a spherical triangle (which also does not lie on the unit sphere) is

$$\mathbf{c} = \frac{\cos^{-1}(\mathbf{v}_1 \cdot \mathbf{v}_2)(\mathbf{v}_1 \times \mathbf{v}_2) + \cos^{-1}(\mathbf{v}_2 \cdot \mathbf{v}_3)(\mathbf{v}_2 \times \mathbf{v}_3) + \cos^{-1}(\mathbf{v}_3 \cdot \mathbf{v}_1)(\mathbf{v}_3 \times \mathbf{v}_1)}{2\Omega|\mathbf{v}_1 \times \mathbf{v}_2||\mathbf{v}_2 \times \mathbf{v}_3||\mathbf{v}_3 \times \mathbf{v}_1|}, \quad (34)$$

and it is implemented is `SphTriCenter()`. Sometimes it is necessary to compute the circumcenter of the spherical triangle (which of course is the same as that of a flat triangle). The function `CircumCenter()` takes three vector arguments and returns

$$\mathbf{c} = (\hat{\mathbf{n}} \cdot \mathbf{v}_1)\hat{\mathbf{n}}, \quad \hat{\mathbf{n}} = [(\mathbf{v}_2 - \mathbf{v}_1) \times (\mathbf{v}_3 - \mathbf{v}_1)]_u. \quad (35)$$

Four unit vectors define a spherical quad. Its area and center of mass can be computed from those of the two spherical triangles that comprise the quad. If the point of intersection between the diagonals is of interest, it may be computed as

$$\mathbf{u} = [(\mathbf{v}_1 \times \mathbf{v}_2) \times (\mathbf{v}_3 \times \mathbf{v}_4)]_u \quad (36)$$

by calling the function `GreatCircleInt()` which takes four vector arguments and returns a vector ($\mathbf{u}$). There is also a function testing whether a point on the unit sphere lies in the interior of a convex spherical polygon with `n.verts` vertices, `InsideWedge()`, which is a member of the `GeoVector` class. The test is performed by checking the signs of the projections of the vector onto the planes defined by each consecutive pair of vertices. The last arguments (`tol`) specifies the acceptable tolerance of this operation. Positive values mean that points that lie on the boundary just outside of the polygon will be treated as if they belong to the interior, while negative values will make boundary points just inside the polygon flagged as exterior.

Some visualization methods might call for a cartographic projection of information on a sphere to a plane. `GeoVector` includes a routine `ProjectToPlane()` that implements gnomonic, orthographic, and equidistant

projections that correspond to the value of the first argument, **projection**, set to 1, 2, and 3, respectively. These projection methods are illustrated in Figure 22. If the point has polar coordinates θ and φ , its projected coordinates on a plane, ξ and η , are given by

$$\begin{aligned}\xi &= \tan \theta \cos \varphi, \quad \eta = \tan \theta \sin \varphi && \text{(gnomonic),} \\ \xi &= \sin \theta \cos \varphi, \quad \eta = \sin \theta \sin \varphi && \text{(orthographic),} \\ \xi &= \theta \cos \varphi, \quad \eta = \theta \sin \varphi && \text{(equidistant).}\end{aligned}\tag{37}$$

The argument **normal** specifies the axis from which the polar angle θ will be measured, and the argument **north** specifies the zero point of the φ angle. The answer is returned in the last two arguments, **xi** and **eta**.

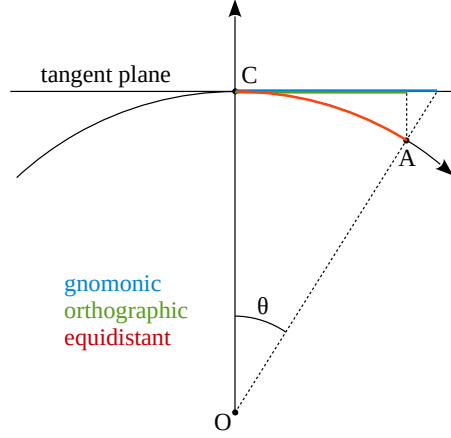


Figure 22: A projection of a point on the surface of a unit sphere using gnomonic (blue), orthographic (green) and equidistant (red) projection methods. The last one projects onto a curved sheet that is subsequently laid flat.

Appendix B: Distance maps used by SPECTRUM

We use a function $[0, 1] \rightarrow [1, \hat{r}_1]$ to map a uniform (index) variable ξ into the physical distance variable $r \in [r_{\min}, r_{\max}]$. For convenience r is normalized to the distance to the inner boundary, $\hat{r} = r/r_{\min}$. The mapping function thus must satisfy $\hat{r}(0) = 1$ and $\hat{r}(1) = \hat{r}_1 = r_{\max}/r_{\min}$. The class **DistanceBase** is the common parent of the distance maps in SPECTRUM. Together with its derived classes it is declared in **distance_map.hh** and the implementation details can be found **distance_map.cc**. Both files reside in the **geometry** directory.

DistanceBase and its derivatives define the map limits, **rmin** and **rmax** that are passed to the class by calling its **Setup()** member function. The function accepts a standard **DataContainer** containing all configuration parameters. Three public interface functions are defined: **GetPhysical()** converts the input **ref** from reference space to physical space, **GetReference()** performs the reverse, and **GetDerivative()** computes the $dr/d\xi$. Some examples of mapping functions are given below.

Exponential distance function

The exponential function has the useful property that the physical distance Δ between any two points equidistant in the reference space is proportional to the distance from the origin (r): $\Delta \sim r$. The map is given by

$$\hat{r} = \hat{r}_1^\xi.\tag{38}$$

The inverse map is

$$\xi = \frac{\ln \hat{r}}{\ln \hat{r}_1}, \quad (39)$$

and the derivative is

$$\frac{d\hat{r}}{d\xi} = \hat{r}_1^\xi \ln \hat{r}_1. \quad (40)$$

It is implemented in `DistanceExponential`. The setup routine has the same arguments as the base class.

Power law distance function

The power law map is specified with a single parameter α as

$$\hat{r} = (1 + B\xi)^\alpha, \quad (41)$$

where $\alpha > 0$. Clearly, this satisfies $\hat{r}(0) = 1$. The remaining constraint gives

$$(1 + B)^\alpha = \hat{r}_1, \quad (42)$$

$$B = \hat{r}_1^{1/\alpha} - 1. \quad (43)$$

Values of $\alpha > 1$ generate a map that is denser near inner boundary, while $\alpha < 1$ result in more points placed closer to the outer boundary; $\alpha = 1$ corresponds to the linear map. Another useful limit is $\alpha \rightarrow \infty$ that produces the exponential map. The inverse map is

$$\xi = \frac{\hat{r}^{1/\alpha} - 1}{B}, \quad (44)$$

while the derivative is

$$\frac{d\hat{r}}{d\xi} = \frac{\alpha(1 + B\xi)^{\alpha-1}}{B}. \quad (45)$$

The class is named `DistancePowerLaw`. The setup routine takes the power law index (`pow1`) as the third argument.

Two-region distance function

This map is suitable for situations where uniformly high radial resolution is required at small r . The formula is

$$\hat{r} = 1 - C + B\xi + Cy^\xi, \quad (46)$$

where y is a parameter. This expression combines the features of an (almost) uniform mesh $1 - C + B\xi$ and the exponential mesh Cy^ξ . We impose one additional constraint, $\hat{r}(a) = \hat{r}_a$, where $0 < a < 1$ and $1 < \hat{r}_a < \hat{r}_1$. This enlarges the parameter space to 3. The constraints yield

$$1 - C + aB + Cy^a = \hat{r}_a, \quad (47)$$

$$1 - C + B + Cy = \hat{r}_1. \quad (48)$$

$$C = \frac{\hat{r}_a - 1 - a(\hat{r}_1 - 1)}{y^a - 1 - a(y - 1)}, \quad (49)$$

$$B = \hat{r}_1 - 1 - C(y - 1). \quad (50)$$

To generate a mesh condensed near the origin one should take a large y and put a large number of points (proportional to aN) between $r_{\min}$ and r_a . In that regime the mesh spacing at the inner boundary is weakly dependent on y , and is essentially $(r_a - r_{\min})/(aN)$. The inverse map is

$$\xi = \chi - \frac{W((C/B)y^\chi \ln y)}{\ln y}, \quad (51)$$

where

$$\chi = \frac{\hat{r} - 1 + C}{B} \quad (52)$$

and W is the Lambert (or product-log) function. The derivative is

$$\frac{d\hat{r}}{d\xi} = B + Cy^\xi \ln y. \quad (53)$$

The map is implemented as the class `DistanceLinearExp`. It takes three additional parameters: `rmed` is the physical distance to be boundary between the uniform and exponential meshing (r_a), `frac` is the fraction of points that fall within the inner region (a), and `logy` specifies the smoothness of the transition ($\ln y$). Larger values of `logy` produce sharper transition between the two regions.

The file `distance_map.cc` implements a testing function `PrintDistanceMaps()` that is enabled by compiling with `-DGE0_DEBUG`. It accepts an object of type `DistanceBase` as its first argument, `dist_map`, and a boolean switch `as_phys` that will make the code print the mesh spacing as a function of physical, instead of reference, distance. The mapping is performed to the physical interval from $r_{\min} = 1$ to $r_{\max} = 100$. The function also checks the consistency between forward and reverse mapping by performing a double conversion. A simple main function code is provided in `main_test_distance_map.cc`. Examples of standard distance maps are shown in Figure 23.

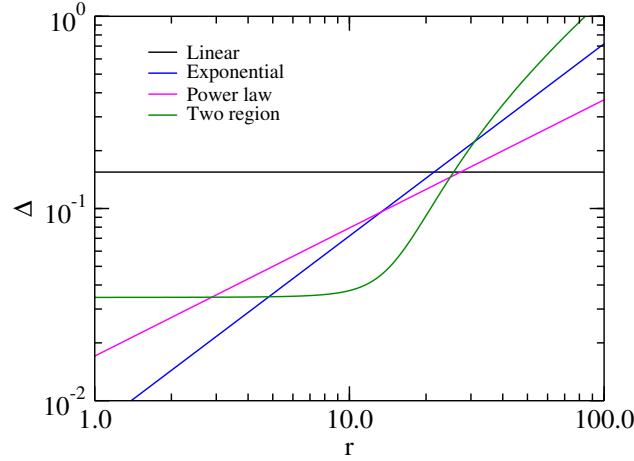


Figure 23: Some examples of distance maps showing the mesh spacing Δ as a function of physical distance r . The following maps are shown: linear (black), exponential (blue), power law with $\alpha = 3$ (magenta) and two-region with $r_a = 10$, $a = 0.4$, and $\ln y = 10$ (green).

Appendix C: Integrals of a linear function over spherical zones

Here we derive expressions for computing the integrals over surface and volume elements of the mesh. Every vector can be written as a product of the radial distance r and a unit vector $\mathbf{r}_u$. Computational zones are addressed by a shell index l and a face index f , t-faces by a shell interface index $l \pm 1/2$ and a face index f , and r-faces by a shell index l and an edge index e . Each grid block stores the radial distances of both shell interfaces $r_{l \pm 1/2}$ and cell midpoints r_l , which are related via

$$r_l = \frac{r_{l-1/2} + r_{l+1/2}}{2}. \quad (54)$$

We also make use of the following definitions:

$$\int_{\varphi_e} d\varphi = \varphi_e, \quad \int_{\varphi_e} \mathbf{r}_u d\varphi = \varphi_e \mathbf{c}_e, \quad (55)$$

$$\int_{\Omega_f} d\Omega = \Omega_f, \quad \int_{\Omega_f} \mathbf{r}_u d\Omega = \Omega_f \mathbf{c}_f, \quad (56)$$

where the arc length φ_e and arc mass center $\mathbf{c}_e$ are give by eqs. (29) and (30), and the face area Ω_f and face mass center $\mathbf{c}_f$ are given by eqs. (33) and (34), respectively, of Appendix A; those are also stored by the grid blocks.

Consider a linear function of coordinates,

$$U(\mathbf{r}) = U_0 + \mathbf{U}_1 \cdot \mathbf{r}. \quad (57)$$

On a geodesic mesh an integral over a surface or volume element can be separated into that over a surface of a unit sphere and in the radial direction. For the shell interface we will use the upper t-face with the index $l + 1/2$. The coordinates used for integrals over a unit sphere will bear a subscript of “u”. The face area on a unit sphere is Ω_f and the edge length on the unit sphere is φ_e . The following notations will be useful:

$$r_{l+1/2} - r_{l-1/2} = \Delta_l, \quad (58)$$

$$r_{l+1/2}^2 - r_{l-1/2}^2 = 2r_l \Delta_l, \quad (59)$$

$$r_{l+1/2}^3 - r_{l-1/2}^3 = 3r_l^2 \Delta_l \left(1 + \frac{\Delta_l^2}{12r_l^2} \right), \quad (60)$$

$$r_{l+1/2}^4 - r_{l-1/2}^4 = 4r_l^3 \Delta_l \left(1 + \frac{\Delta_l^2}{4r_l^2} \right). \quad (61)$$

Average over a t-face

The area of a t-face is

$$A_{l-1/2,f} = r_{l-1/2}^2 \Omega_f \quad (62)$$

(the expression is written for the bottom face of a zone with a shell index l ; to apply to the top face replace $l - 1/2$ with $l + 1/2$). Averaging (57) over the t-face yields

$$\begin{aligned} \langle U \rangle_{l-1/2,f} &= \frac{1}{A_{l-1/2,f}} \int_{A_{l-1/2,f}} (U_0 + \mathbf{U}_1 \cdot \mathbf{r}) dA = \frac{1}{r_{l-1/2}^2 \Omega_f} \int_{\Omega_f} [U_0 + r_{l-1/2} (\mathbf{U}_1 \cdot \mathbf{r}_u)] r_{l+1/2}^2 d\Omega \\ &= U_0 + r_{l-1/2} (\mathbf{U}_1 \cdot \mathbf{c}_f) = U(r_{l-1/2} \mathbf{c}_f), \end{aligned} \quad (63)$$

Equation (63) shows that the average of a linear function over the t-face is exactly equal to its value at the mass center of the t-face. This is true in general for any shape.

Average over an r-face

The area of an r-face is

$$A_{le} = \frac{(r_{l+1/2}^2 - r_{l-1/2}^2) \varphi_e}{2} = r_l \Delta_l \varphi_e. \quad (64)$$

Averaging (57) over the r-face yields

$$\begin{aligned} \langle U \rangle_{le} &= \frac{1}{A_{le}} \int_{A_{le}} (U_0 + \mathbf{U}_1 \cdot \mathbf{r}) dA = \frac{1}{r_l \Delta_l \varphi_e} \int_{\varphi_e} \int_{r_{l-1/2}}^{r_{l+1/2}} [U_0 + r (\mathbf{U}_1 \cdot \mathbf{r}_u)] r dr d\varphi \\ &= \frac{1}{r_l \Delta_l \varphi_e} \left[\frac{(r_{l+1/2}^2 - r_{l-1/2}^2) \varphi_e U_0}{2} + \frac{(r_{l+1/2}^3 - r_{l-1/2}^3) \varphi_e (\mathbf{U}_1 \cdot \mathbf{c}_e)}{3} \right] \\ &= U_0 + r_l \left(1 + \frac{\Delta_l^2}{12r_l^2} \right) (\mathbf{U}_1 \cdot \mathbf{c}_e) = U(\bar{r}_s \mathbf{c}_e), \end{aligned} \quad (65)$$

where

$$\bar{r}_l = r_l \left(1 + \frac{\Delta_l^2}{12r_l^2} \right). \quad (66)$$

As expected, a linear function is averaged by evaluating its value at the mass center of the r-face.

Average over the frustum

The volume of the frustum is

$$V_{lf} = \frac{(r_{l+1/2}^3 - r_{l-1/2}^3)\Omega_f}{3} = r_l \bar{r}_l \Delta_l \Omega_f. \quad (67)$$

Averaging (57) over the frustum yields

$$\begin{aligned} \langle U \rangle_{lf} &= \frac{1}{V_{lf}} \int_{V_{lf}} (U_0 + \mathbf{U}_1 \cdot \mathbf{r}) dV = \frac{1}{r_l \bar{r}_l \Delta_l \Omega_f} \int_{\Omega_f} \int_{r_{l-1/2}}^{r_{l+1/2}} [U_0 + r(\mathbf{U}_1 \cdot \mathbf{r}_u)] r^2 dr d\Omega \\ &= \frac{1}{r_l \bar{r}_l \Delta_l \Omega_f} \left[\frac{(r_{l+1/2}^3 - r_{l-1/2}^3)\Omega_f U_0}{3} + \frac{(r_{l+1/2}^4 - r_{l-1/2}^4)\Omega_f (\mathbf{U}_1 \cdot \mathbf{c}_f)}{4} \right] \\ &= U_0 + r_l \left(1 + \frac{\Delta_l^2}{4r_l^2} \right) \left(1 + \frac{\Delta_l^2}{12r_l^2} \right)^{-1} (\mathbf{U}_1 \cdot \mathbf{c}_f) = U(\tilde{r}_l \mathbf{c}_f), \end{aligned} \quad (68)$$

where

$$\tilde{r}_l = r_l \left(1 + \frac{\Delta_l^2}{4r_l^2} \right) \left(1 + \frac{\Delta_l^2}{12r_l^2} \right)^{-1}. \quad (69)$$

A linear function is averaged by taking its value at the center of mass of the frustum.

Appendix D: Reconstruction in exponential normal coordinates

Most distance maps (Appendix B) produce cells with variable aspect ratio (the ratio between the lengths of an r-edge and a t-edge) that depends on the shell index. The only exception to this rule is the exponential distance function given by eq. (38). This means that all operations like volume averaging are identical for all values of the radial index l . To make this useful we work with *two* distance maps, the actual one ($\xi \rightarrow r$) and the exponential ($\xi \rightarrow r'$). To eliminate the dependence of the moments on the shell index, reconstruction is performed in exponential normal coordinates (ENCs) that are normalized to the exponential distance to the shell mid-point $\bar{r}'_l$. All ENC geometric quantities will bear a breve, e.g., $\check{\mathbf{r}} = \mathbf{r}' / \bar{r}'_l$.

We will perform reconstruction in ENCs ($\check{\mathbf{r}}$) instead of physical coordinates ($\mathbf{r}$). The the zone average (18) can be written as

$$\langle U \rangle_{lf} = \frac{1}{r_l \bar{r}_l \Delta_l \Omega_f \Lambda} \int_{\Omega_f} \int_{r'_{l-1/2}}^{r'_{l+1/2}} W(\mathbf{r}') r'^2 dr' d\Omega = \frac{r_l'^3}{r_l \bar{r}_l \Delta_l \Omega_f \Lambda} \int_{\Omega_f} \int_{1-\check{\Delta}/2}^{1+\check{\Delta}/2} W(\check{\mathbf{r}}) \check{r}^2 d\check{r} d\Omega, \quad (70)$$

where $\Lambda = \ln(r_{\max}/r_{\min})$, W is given by eq. (20), and we use scalar notation for U and W to avoid confusion with the linear modes $\delta \mathbf{W}$ which comprise a 3-vector. The quantity

$$\check{\Delta} = \frac{\Delta'_l}{r'_l} \quad (71)$$

is independent of l , so the radial dependence is only present in the constant term in front of the integral in (70). The integral is related to the average of W over the zone,

$$\langle W \rangle_{lf} = \frac{1}{\check{\Delta}(1 + \check{\Delta}^2/12)\Omega_f} \int_{\Omega_f} \int_{1-\check{\Delta}/2}^{1+\check{\Delta}/2} W(\check{\mathbf{r}}) \check{r}^2 d\check{r} d\Omega = \frac{r_l \bar{r}_l \Delta_l \Lambda}{r_l'^3 \check{\Delta}(1 + \check{\Delta}^2/12)} \langle U \rangle_{lf}. \quad (72)$$

We will reconstruct $W(\check{\mathbf{r}})$ instead of $U(\mathbf{r})$, and then perform the conversion. This means that U itself is *not* a linear function of coordinates $\mathbf{r}$, but has some complicated nonlinear behavior. However this is not of concern because the reconstruction is applied to a set of neighbor cells, so that the corrections to the linear map are small in magnitude. A linear reconstruction of W on a stencil s is

$$W_{lf}^s(\check{\mathbf{r}}) = \langle W \rangle_{lf} + \delta \mathbf{W}_{lf}^s \cdot (\check{\mathbf{r}} - \langle \check{\mathbf{r}} \rangle_f), \quad (73)$$

where

$$\langle \check{\mathbf{r}} \rangle_f = \frac{1}{\check{\Delta}(1 + \check{\Delta}^2/12)\Omega_f} \int_{\Omega_f} \int_{1-\check{\Delta}/2}^{1+\check{\Delta}/2} \check{\mathbf{r}} \check{r}^2 d\check{r} d\Omega = \frac{1 + \check{\Delta}^2/4}{1 + \check{\Delta}^2/12} \mathbf{c}_f. \quad (74)$$

The linear modes $\delta \mathbf{W}_{lf}^s$ are found by integrating (73) over each zone in the stencil. Recall that the principal zone is not included in the stencil, and we can see why – the integral over it gives an identity. Suppose the stencil zones are indexed as (l', f') , where we typically have $f' = f$ or $f' = f \pm 1$. Let's evaluate the center of mass of the zone $(l + 1, f')$:

$$\begin{aligned} \langle \check{\mathbf{r}} \rangle_{+f'} &= \frac{1}{\check{\Delta}_+(1 + \check{\Delta}_+^2/12)\Omega_{f'}} \int_{\Omega_{f'}} \int_{1+\check{\Delta}/2}^{1+\check{\Delta}/2+\check{\Delta}_+} \check{\mathbf{r}} \check{r}^2 d\check{r} d\Omega = \frac{\mathbf{c}_{f'}}{\check{\Delta}_+(1 + \check{\Delta}_+^2/12)\Omega_{f'}} \int_{1+\check{\Delta}/2}^{1+\check{\Delta}/2+\check{\Delta}_+} \check{r}^3 d\check{r} \quad (75) \\ &= \frac{\mathbf{c}_{f'}}{\check{\Delta}_+(1 + \check{\Delta}_+^2/12)(1 + \check{\Delta}/2 + \check{\Delta}_+/2)^4} \int_{1+\check{\Delta}/2}^{1+\check{\Delta}/2+\check{\Delta}_+} \check{r}^3 d\check{r}. \end{aligned}$$

Averaging (73) over the stencil zone (l', f') yields

$$\langle W_{l'f'} \rangle = \langle W \rangle_{lf} + \delta \mathbf{W}_{lf}^s \cdot (\check{\mathbf{r}} - \langle \check{\mathbf{r}} \rangle_f), \quad (76)$$